ADC
ANALOG TO DIGITAL CONVERTER

Presented by: Group-2
MD Fayed Hasan (2210176143)
MD Shamiur Rahman Sefat (2110276105)
Momin Sarker (2110676137)

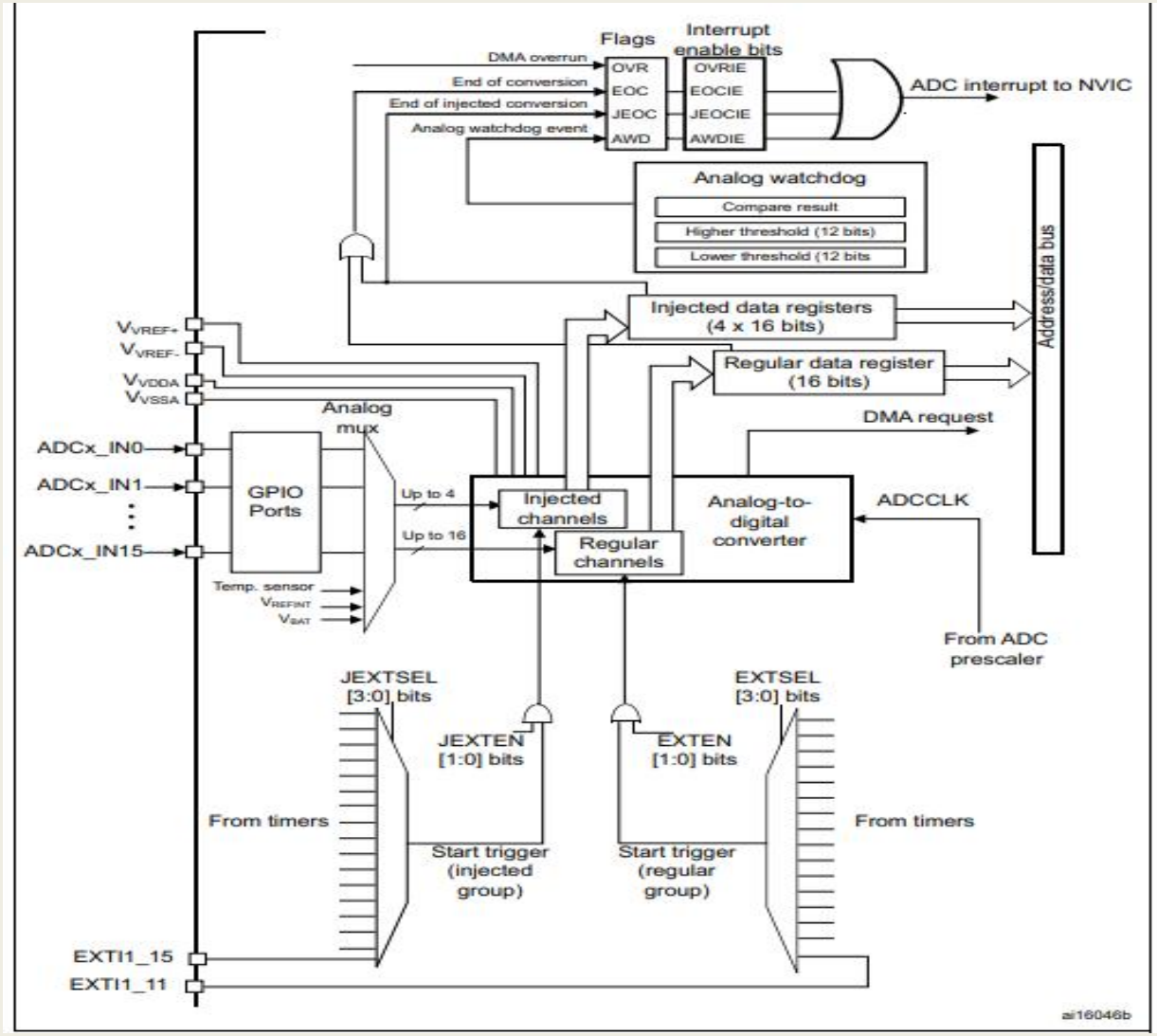
ADC introduction

- The 12-bit ADC is a successive approximation analog-to-digital converter. It has up to 19 multiplexed channels allowing it to measure signals from 16 external sources, two internal sources, and the VBAT channel. The A/D conversion of the channels can be performed in single, continuous, scan or discontinuous mode. The result of the ADC is stored into a leftor right-aligned 16-bit data register.

ADC main features

- STM32F446 ADC Features
- Three independent 12-bit ADCs
- Up to 24 external channels
- Resolution:
  - *12-bit*
  - *10-bit*
  - *8-bit*
  - *6-bit*
- Regular conversion
- Injected conversion
- Scan mode
- Continuous conversion
- Discontinuous mode
- DMA support
- Analog watchdog
- External trigger support

ADC Block Diagram



ADC Conversion Process

- Select ADC channel
- Configure sampling time
- Start conversion
- Sample analog voltage
- Convert to digital
- Store result in ADC_DR
- Read by CPU or DMA

ADC clock

The ADC features two clock schemes:

1.Clock for the analog circuitry: ADCCLK, common to all ADCs This clock is generated from the APB2 clock divided by a programmable prescaler that allows the ADC to work at fPCLK2/2, /4, /6 or /8. Refer to the datasheets for the maximum value of ADCCLK.

2Clock for the digital interface (used for registers read/write access) This clock is equal to the APB2 clock. The digital interface clock can be enabled/disabled individually for each ADC through the RCC APB2 peripheral clock enable register (RCC_APB2ENR).

Single conversion mode

- Once the conversion of the selected channel is complete:
- If a regular channel was converted:
  - The converted data are stored into the 16-bit ADC_DR register
  - The EOC (end of conversion) flag is set
  - An interrupt is generated if the EOCIE bit is set
- If an injected channel was converted:
  - The converted data are stored into the 16-bit ADC_JDR1 register
  - The JEOC (end of conversion injected) flag is set
  - An interrupt is generated if the JEOCIE bit is set Then the ADC stops.

Continuous conversion mode

- In continuous conversion mode, the ADC starts a new conversion as soon as it finishes one. This mode is started with the CONT bit at 1 either by external trigger or by setting the SWSTART bit in the ADC_CR2 register (for regular channels only).
- After each conversion:
    - If a regular group of channels was converted:
        - The last converted data are stored into the 16-bit ADC_DR register
        - The EOC (end of conversion) flag is set
        - An interrupt is generated if the EOCIE bit is set

Multi ADC mode

- In devices with two ADCs or more, the Dual (with two ADCs) and Triple (with three ADCs) ADC modes can be used
- The four possible modes below are implemented:
    - Injected simultaneous mode
    - Regular simultaneous mode
    - Interleaved mode
    - Alternate trigger mode

ADC registers

- ADC status register (ADC_SR)
- ADC control register 1 (ADC_CR1)
- ADC control register 2 (ADC_CR2)
- ADC sample time register 1 (ADC_SMPR1)
- ADC sample time register 2 (ADC_SMPR2)
- ADC injected channel data offset register x (ADC_JOFRx) (x=1..4)
- ADC watchdog higher threshold register (ADC_HTR)
- ADC watchdog lower threshold register (ADC_LTR)
- ADC regular sequence register 1 (ADC_SQR1)
- ADC regular sequence register 2 (ADC_SQR2)
- ADC regular sequence register 3 (ADC_SQR3)
- ADC injected sequence register (ADC_JSQR)
- ADC injected data register x (ADC_JDRx) (x= 1..4)
- ADC regular data register (ADC_DR)
- ADC Common status register (ADC_CSR)
- ADC common control register (ADC_CCR)

ADC Applications

- Temperature Monitoring
- Battery Voltage Measurement
- Medical Devices
- Robotics
- Industrial Automation
- IoT Sensors
- Motor Control
- Audio Processing

Advantages

- High accuracy
- Multiple channels
- DMA support
- Flexible trigger sources
- Multiple resolutions
- Fast conversion

THANK YOU

CRC
CYCLIC REDUNDANCY CHECK

PRESENTED BY

RUBAIYET E KABIR (2110876147)
MD NADIM (2110276119)
MD IYASIN UL ALAM (2210976123)

INTRODUCTION

A Cyclic Redundancy Check (CRC) in microcontroller programming is an error-detecting code used to spot accidental changes in data during communication or memory storage. It uses a generator polynomial to produce a short checksum value matched by the sender and receiver.

REGISTERS OF CRC

01. Data Register

02. Independent Data Register

03. Control Register

FEATURES

01

Uses CRC-32 (Ethernet) polynomial: 0x04C11DB7

02

Single input/output 32-bit data register

03

CRC computation done in four AHB clock cycles (HCLK)

04

General-purpose 8-bit register

WORK FLOW

AHB (ADVANCED HIGH PERFORMANCE) BUS

DATA REGISTER (INPUT)

CRC COMPUTATION LOGIC

DATA REGISTER (OUTPUT)

POLYNOMIAL EQU.

$$X32 + X26 + X23 + X22 + X16 + X12 + X11 + X10 + X8 + X7 + X5 + X4 + X2 + X + 1$$

THANK YOU VERY MUCH!

STM32 MICROCONTROLLER COURSE

CRC Calculation Unit

Hardware-Accelerated Cyclic Redundancy Check on STM32

GROUP 9

2210776130	**Md Shibli Noman Sarkar**
2210776147	**Md Ruhul Amin**
2210776132	**Md Mehedi Hasan Rafy**

What Is CRC?

Definition

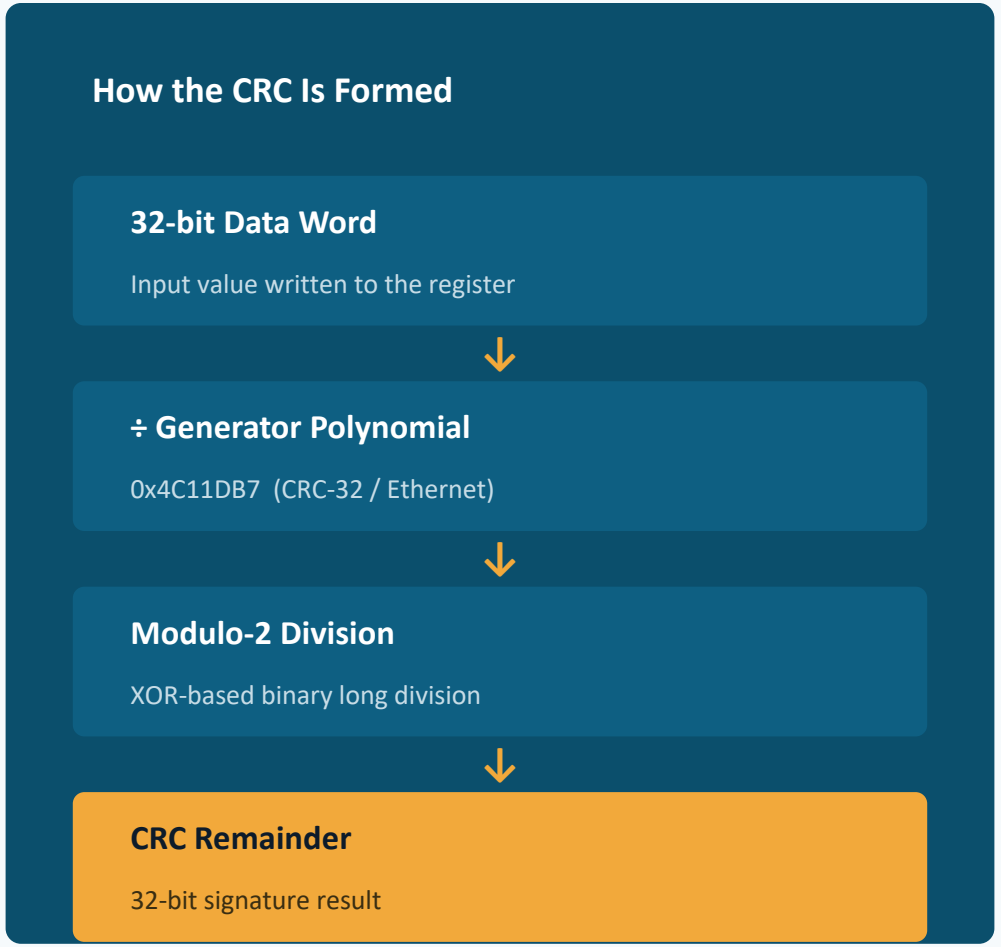
The CRC calculation unit derives a fixed-length CRC code from a 32-bit data word using a fixed generator polynomial — a hardware checksum engine built into the STM32.

The Theory

- Data is treated as a binary polynomial and divided by a fixed generator polynomial using modulo-2 (XOR) arithmetic.
- The remainder of that division is the CRC signature — a compact fingerprint of the data.
- Any bit change in the data almost certainly changes the remainder, so mismatched CRCs reveal corruption.

Why it matters

Verifies data transmission & storage integrity, and checks flash memory contents against the EN/IEC 60335-1 safety standard.



CRC Main Features & Block Diagram

1 **Polynomial**

Uses the CRC-32 (Ethernet) polynomial: 0x4C11DB7

2 **Registers**

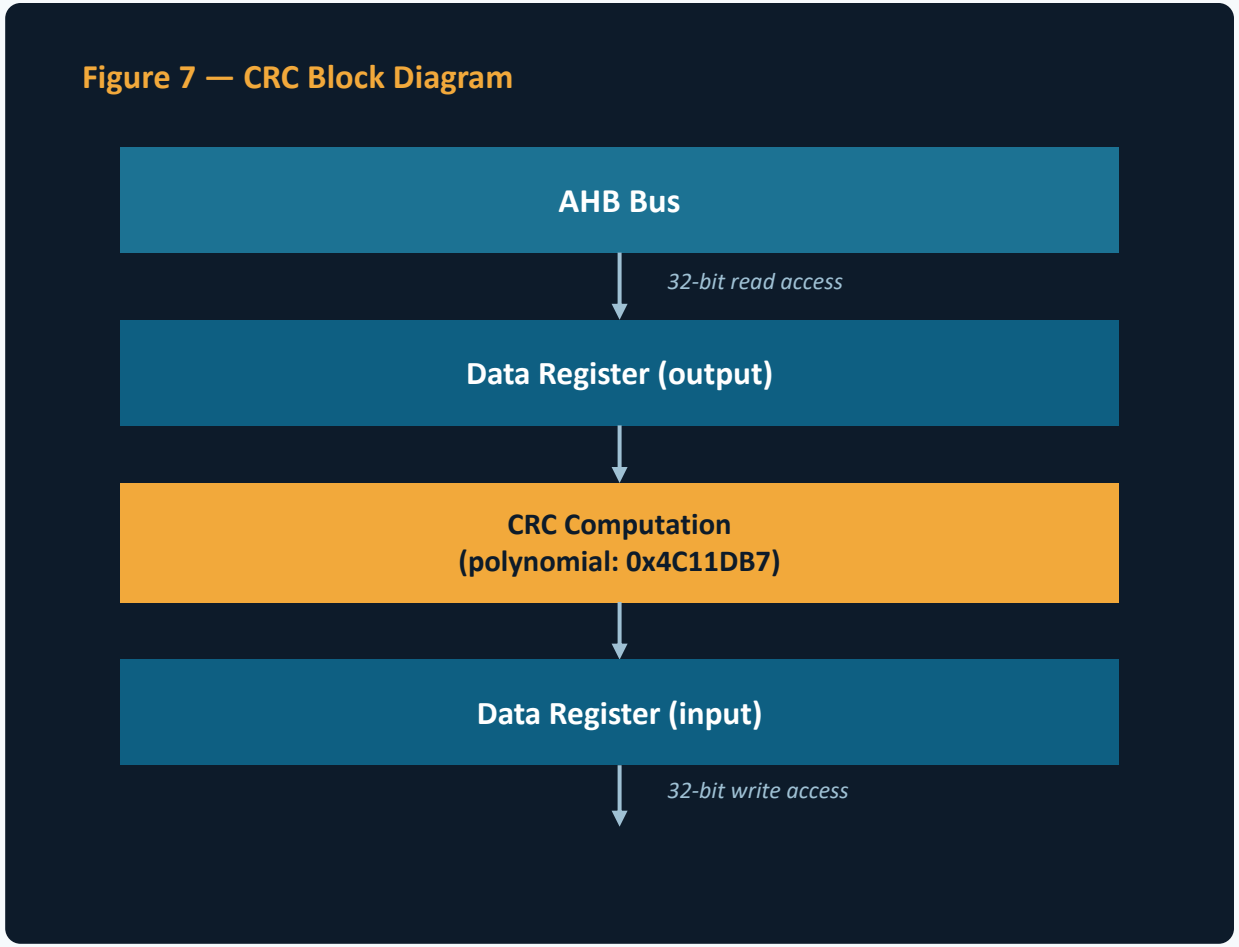
Single 32-bit input/output data register

3 **Speed**

CRC computation completed in just 4 AHB clock cycles (HCLK)

4 **Bonus Storage**

Extra general-purpose 8-bit register for temporary data



How the CRC Unit Operates

The CRC calculation unit mainly consists of a single 32-bit data register, which is used to enter new data and holds the result of the previous calculation when read.

1

Write to Combine

Every write into the data register combines the previous CRC value with the new data — the whole 32-bit word is processed at once, not byte-by-byte.

2

Stall Until Done

A write is stalled until CRC computation finishes, so back-to-back writes or a write followed by a read work safely without extra handshaking.

3

Reset to 0xFFFFFFFF

Setting the RESET bit in CRC_CR reinitializes the calculator to 0xFFFF FFFF. The independent CRC_IDR register is left untouched.

Note: The CRC unit contains two data registers and a control register — all accessed by full 32-bit words.

Data Register & Independent Data Register

CRC_DR — Data Register

Offset 0x00

Reset value: 0xFFFF FFFF



rw · read/write, all 32 bits

Function

- Used as the input register to enter new data into the CRC calculator on write.
- Holds the result of the previous CRC calculation when read back.

CRC_IDR — Independent Reg.

Offset 0x04

Reset value: 0x0000 0000



rw · only bits 7:0 usable

Function

- General-purpose 8-bit register — free temporary byte storage, unrelated to the CRC math.
- Not affected by a CRC RESET, so scratch data survives a CRC recalculation.

Control Register & Register Map

CRC_CR — Control Register

Offset 0x08

Reset value: 0x0000 0000

Reserved [31:1]	RESET

Bit 0 — RESET

- Resets the CRC calculation unit and sets the data register to 0xFFFF FFFF.
- Write-only: this bit can only be set by software; hardware clears it automatically.

Table 13 — Register Map & Reset Values

Offset	Register	Reset Value
0x00	CRC_DR — Data register	0xFFFF FFFF
0x04	CRC_IDR — Independent data register	0x0000 0000
0x08	CRC_CR — Control register	0x0000 0000

Access Rule

All three CRC registers must be accessed as full 32-bit words — the peripheral does not support byte or half-word access. Reserved bits always read as 0 and must be kept at their reset value on writes.

STM32's Role in Data Integrity

STM32 doesn't just support CRC in software — it embeds a dedicated hardware CRC-32 engine directly on the AHB bus, turning a slow software task into a near-instant peripheral operation.

4

AHB CLOCK CYCLES

to compute a full 32-bit CRC in hardware — versus hundreds of CPU cycles for an equivalent software routine.

1

Firmware Integrity

Computes a runtime signature of the software, compared against a reference generated at link-time and stored in memory.

2

Safety Compliance

Provides the flash-memory integrity check required by the EN/IEC 60335-1 household-appliance safety standard.

3

CPU Offload

Frees the processor core entirely — the CRC engine runs independently on the AHB bus while the CPU does other work.

4

Communication Checks

Validates data received over UART, CAN, SPI, or storage links, catching transmission errors before they matter.

Thank You

Questions & Discussion Welcome

GROUP 9

Md Shibli Noman Sarkar · Md Ruhul Amin · Md Mehedi Hasan Rafy

MICROCONTROLLER INTERFACING

Digital-to-Analog Converter (DAC)

of STM32F446RE Nucleo

GROUP NO. 13

2210776138 — Md Majnu Miah

2210776118 — Plabon Dey

2210976133 — Md Maruf E Elahi

12-BIT • TWO CHANNELS • DMA

From digital code to analog voltage

- 12-bit voltage-output converter; configurable for 8- or 12-bit operation
- 12-bit data supports left or right alignment
- Two independent output channels and simultaneous dual conversion
- VREF+ defines the positive reference; DMA support is available





A discrete input code is translated into a proportional output level.

DAC main features

01 12-bit output
Two channels

02 8 / 12-bit
Left / right aligned

03 Synchronized
update

04 Noise-wave
generation

05 Triangle-wave
generation

06 Independent or simultaneous
dual conversion

07 DMA per channel
Underrun detection

08 External trigger
selection

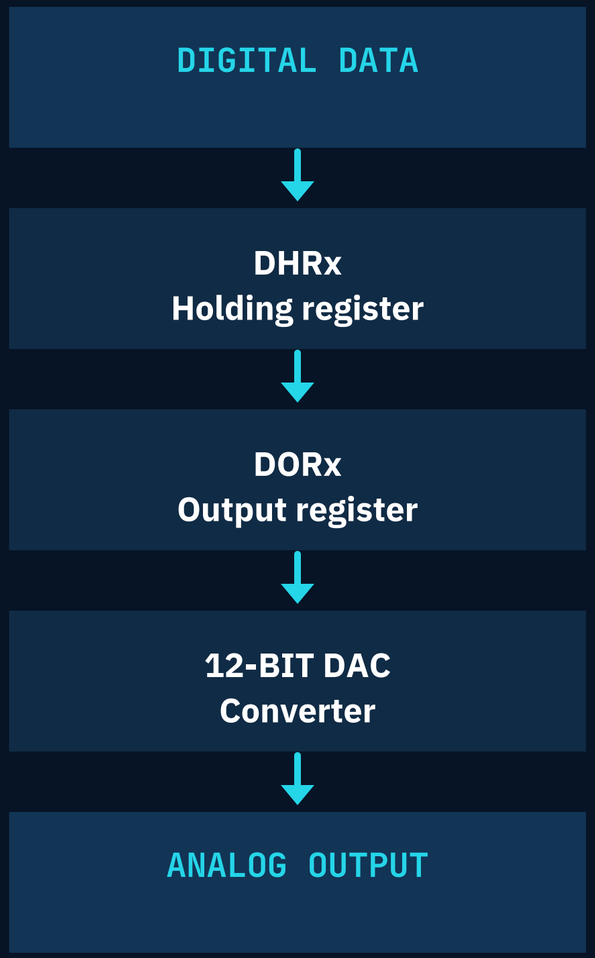
09 VREF+ voltage
reference

Two converters, two buffered outputs



- Each channel owns a converter
- Buffer lowers output impedance and may be disabled
- PA4 / PA5 expose DAC_OUT1 / DAC_OUT2 when enabled

Registers stage every conversion



Software writes conversion data to DAC_DHRx.

DAC_DORx is read-only; the selected DHR value is copied here.

Analog output becomes valid after converter settling time.

```
NO TRIGGER  ·  DHRx → DORx after 1 APB1 cycle
HARDWARE TRIGGER  ·  Trigger → update after 3 APB1 cycles
```

Choose precision and alignment

8-bit Right Alignment

DAC_DHR8Rx

DATA BITS [7:0]

12-bit Left Alignment

DAC_DHR12Lx

DATA BITS [15:4]

12-bit Right Alignment

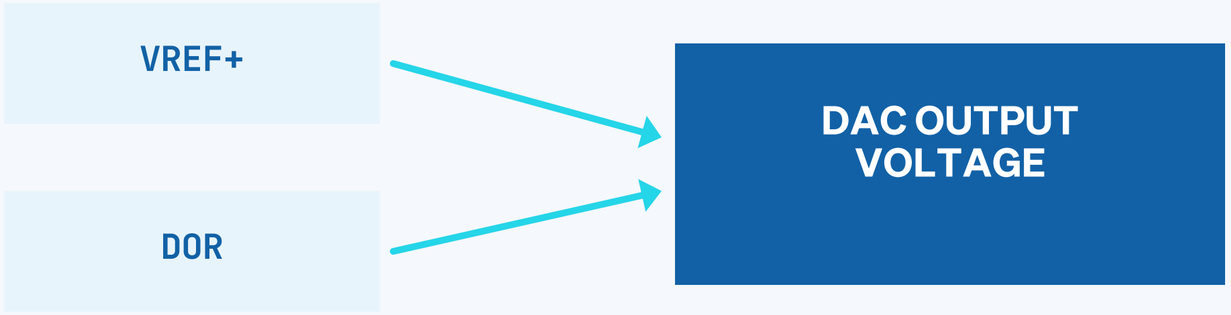
DAC_DHR12Rx

DATA BITS [11:0]



A linear relationship to VREF+

$$\text{DAC output} = (\text{VREF+} \times \text{DOR}) / 4096$$



Digital input codes map linearly to voltages from 0 to VREF+. The final output depends directly on the DAC output register value.

Triggers define exactly when output updates

Trigger Enable
TENx

↓

Trigger Selection
TSELx

↓

Trigger Event

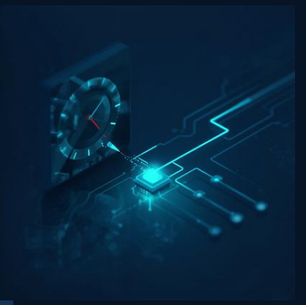
↓

DHRx → DORx

↓

DAC Output

Timer 6 TRGO	Timer 8 TRGO
Timer 7 TRGO	Timer 5 TRGO
Timer 2 TRGO	Timer 4 TRGO
EXTI Line 9	Software Trigger



TSELx[2:0] selects the trigger source

Automated transfers—with underrun protection



- Each DAC channel supports DMA
- DMAENx enables requests on external triggers
- Requests are not queued; a new trigger during service causes underrun
- DMAUDRx reports the error; conversion continues with old data

TRIGGER RATE TOO HIGH → DMA MISSES DEADLINE → UNDERRUN → DMAUDRx SET

Wave generation and dual-channel control

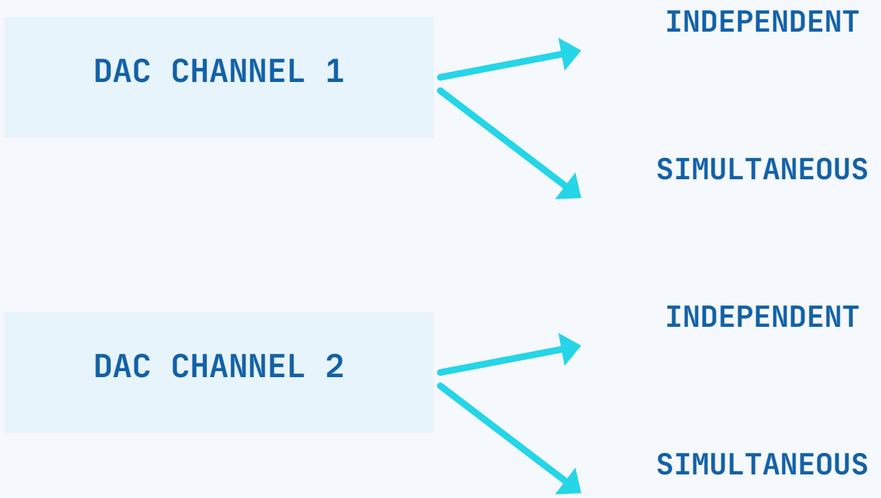
WAVE GENERATION

NOISE GENERATION
WAVE = “01” · LFSR
Trigger must be enabled

TRIANGLE GENERATION
WAVE = “10” · amplitude via MAMP
Triggered internal triangle counter

`DAC_DHRx + WAVE VALUE → DAC_DORx`

DUAL DAC CONVERSION



DUAL REGISTERS · DHR8RD / DHR12RD / DHR12LD
Single access updates both channels

CONCLUSION

One flexible analog-output subsystem



The STM32F446RE DAC provides flexible digital-to-analog conversion through two channels, configurable data formats, trigger control, DMA capability, and wave-generation modes.

THANK YOU

Questions?

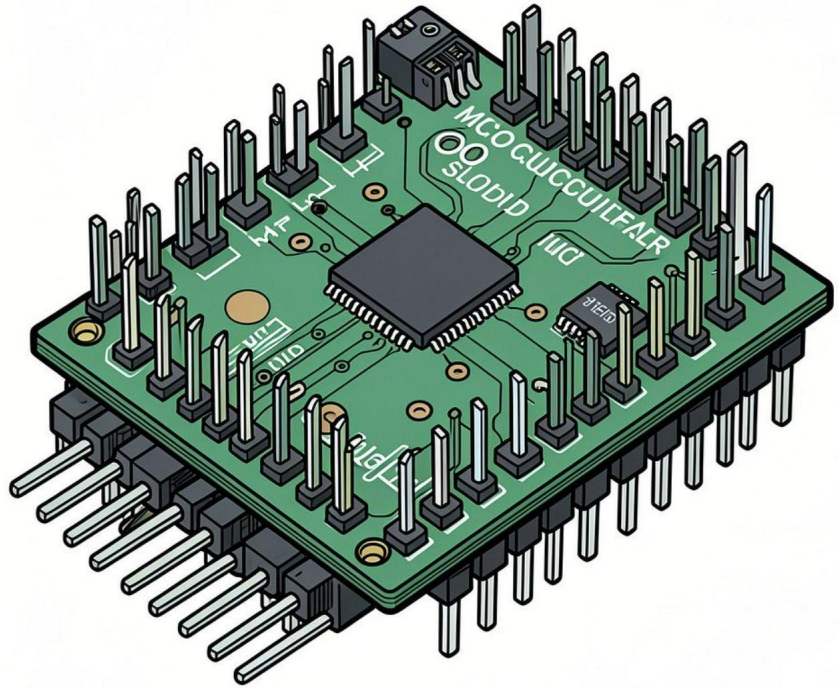
GPIO

Course: Microcontroller and Interfacing

Group: 15(GPIO)

Group Members:

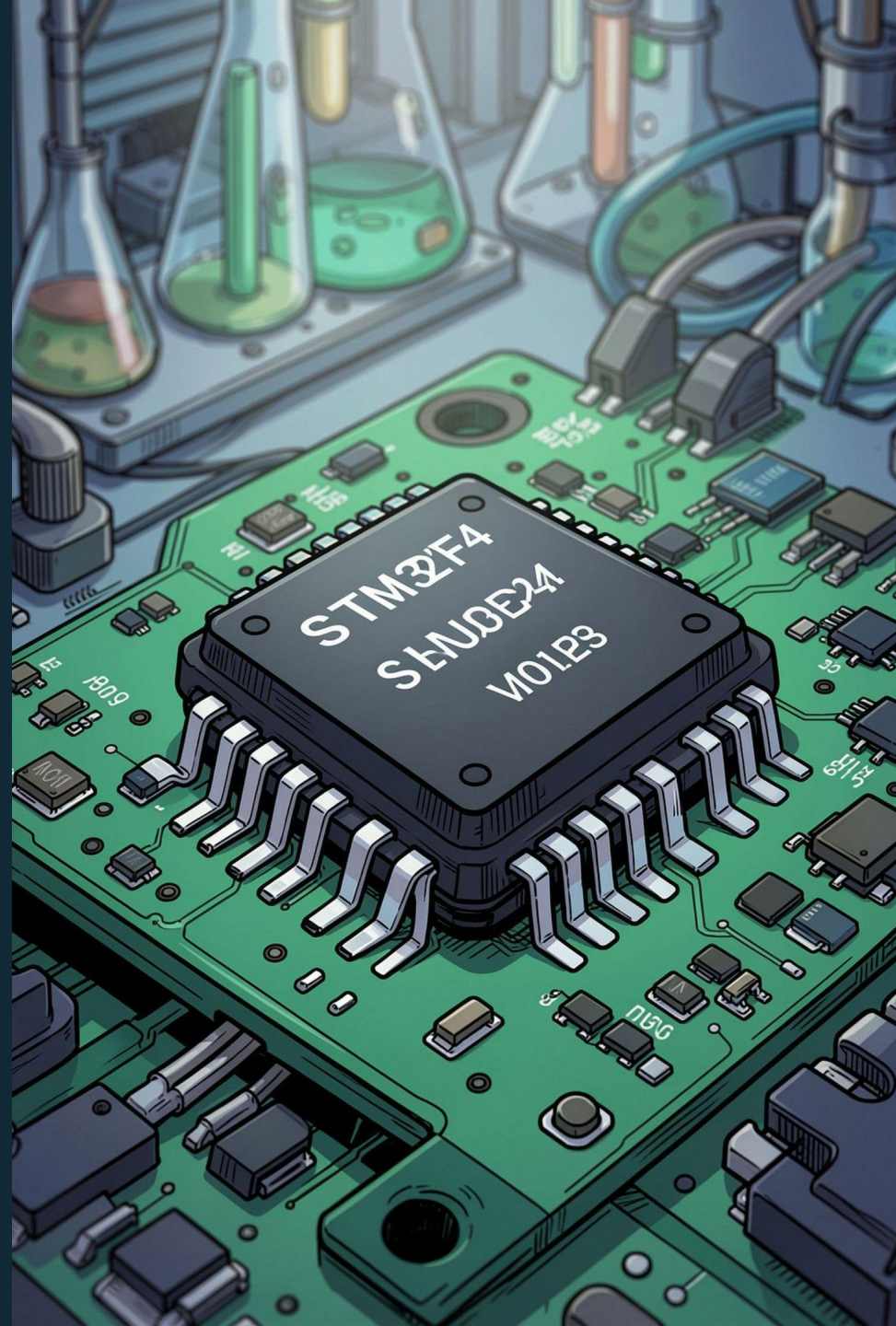
1. Masud Rana (2211176104)
2. Samudra Mazumder (2211176134)
3. Sumayta Rahman (2212276116)



GPIO Architecture of STM32F446

GPIO (General-Purpose Input/Output) is the configurable digital interface that lets the STM32F446 CPU and peripherals interact with external circuits. A single physical pin can be configured to sample signals, drive outputs, or hand control to an internal peripheral — making GPIO both flexible and central to embedded design.

This deck explains the STM32F446 GPIO signal path, register-level controls, electrical options (pulls, output type, speed), alternate functions, atomic set/reset, and practical patterns for reliable interfacing and interrupt-driven designs.



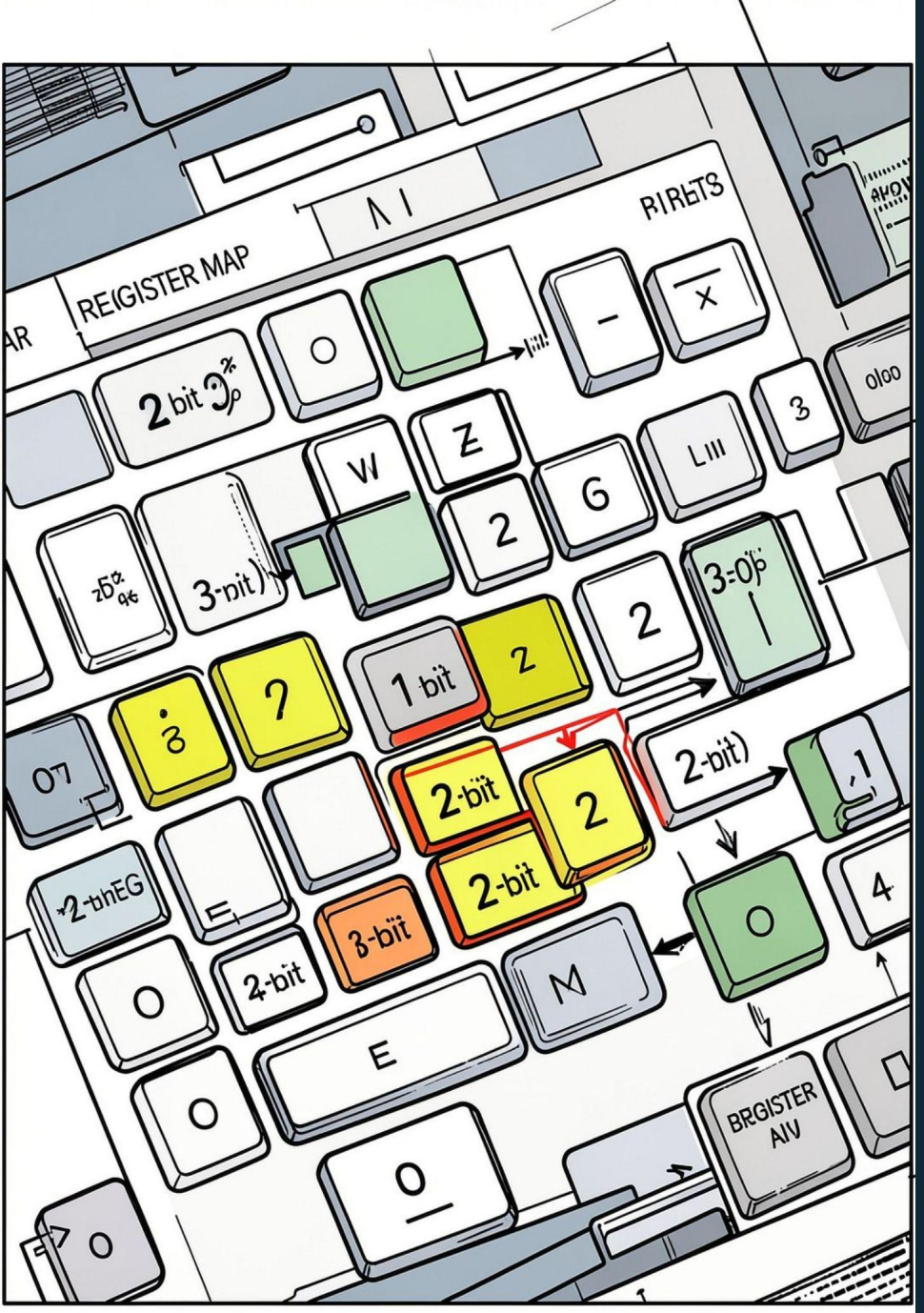
STM32 GPIO Signal Path — Conceptual Overview

The GPIO signal path has two complementary flows that meet at the physical pin. On the output side, software writes to the GPIO output data path, which is then translated by the output driver into a logic-level voltage or current on the pad. On the input side, the external world drives the pin, and the signal is first conditioned by the pad circuitry before being sampled by the input buffer and made visible to the CPU through the input data register.

Output path: CPU write → ODR/BSRR → output latch → output driver → pin pad → external device

Input path: external signal → pin pad → protection network → input buffer / Schmitt trigger → IDR → CPU read

The output path is usually controlled through the output data register (ODR) or the bit set/reset register (BSRR). BSRR is preferred for atomic pin updates because it avoids read-modify-write hazards and lets firmware set and reset different pins in a single write. The output driver itself is responsible for sourcing or sinking current, so its behavior depends on whether the pin is configured as push-pull or open-drain, and whether the output speed setting is optimized for low-noise transitions or faster edges.



GPIO Modes and the MODER Register

Each pin's mode is encoded in MODER with two bits: MODER[2n+1:2n]. The four modes:

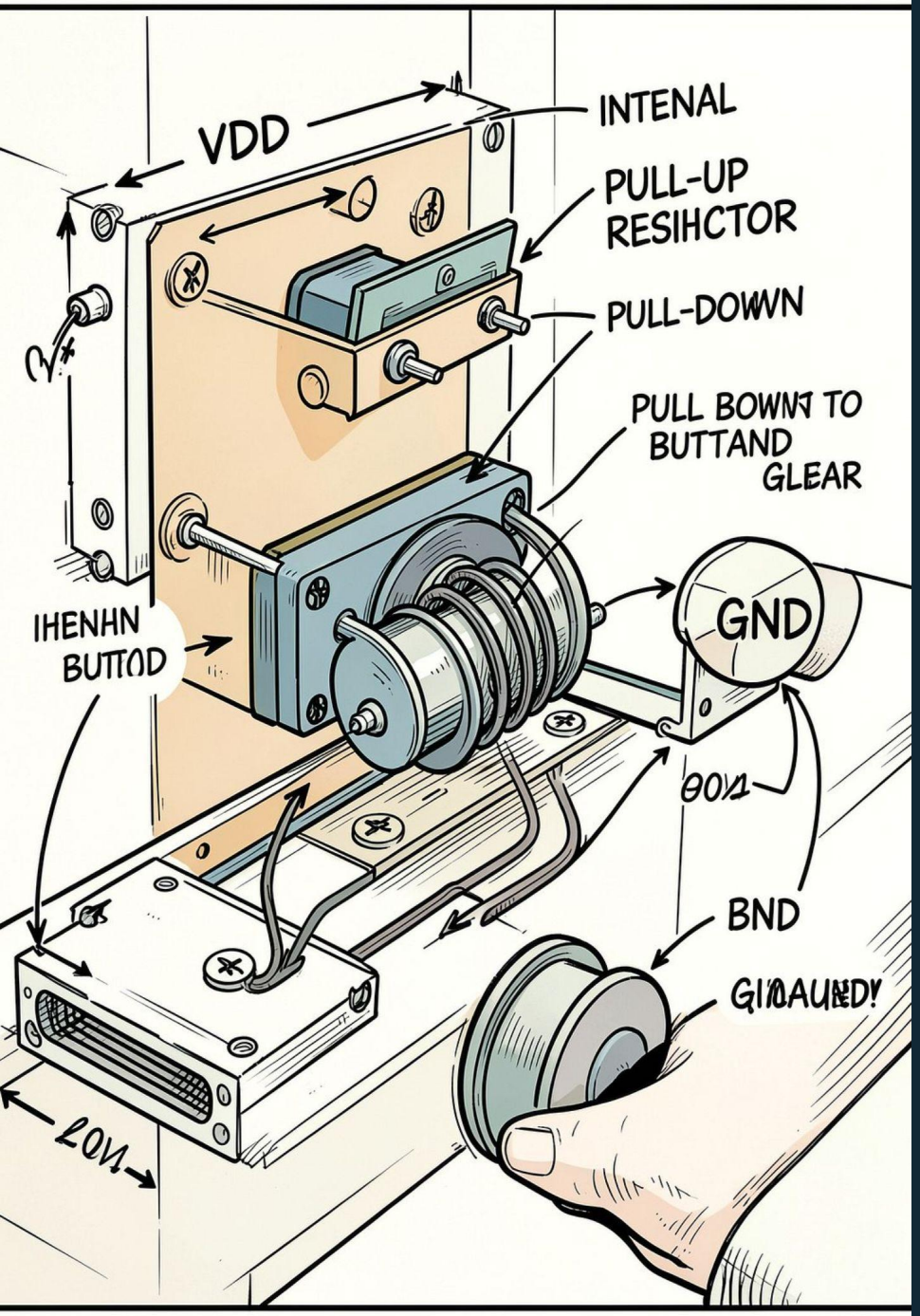
- 00 — Input: digital input, input buffer active
- 01 — General Purpose Output: ODR/BSRR control, output driver enabled
- 10 — Alternate Function: peripheral drives the pin via AFR
- 11 — Analog: analog/low-power (input buffer disabled)

Example: PA5 uses MODER[11:10]. After reset pins default to safe states (typically input with pull-down/up determined by hardware) to avoid driving unknown loads on connected circuits.

GPIO Inputs — IDR and Electrical Considerations

When configured as input, the pin's voltage passes through the protection network and input buffer before being sampled by the IDR register. IDR[n] reads 0 = LOW and 1 = HIGH. Inputs do not drive the line, so the external circuit must establish a valid logic level.

GPIO input pins are designed to present very high input impedance, which minimizes DC loading on the source signal. Even so, the pin and its package trace add capacitance, so fast edges can be slowed and noisy or high-impedance sources may be more vulnerable to distortion. For reliable operation, keep source impedance low enough to charge the input capacitance quickly, and use buffering, series resistance, or RC filtering when the signal needs conditioning before it reaches the MCU.



Pull-Up / Pull-Down — PUPDR Register

Each pin has two PUPDR bits (PUPDR[2n+1:2n]) with values:

- 00 — No pull
- 01 — Pull-up (weak)
- 10 — Pull-down (weak)
- 11 — Reserved

Internal pulls provide a weak bias to define default logic states, reducing BOM by avoiding external resistors for simple inputs. Choose pull direction based on your external circuit (e.g., pull-up for active-low buttons to GND).

GPIO Output: ODR, BSRR, and Atomic Control

Output state is stored in ODR (ODR[n] = 1 → HIGH, 0 → LOW). For atomic single-pin writes, use BSRR:

- BSRR[0:15] — set pins corresponding to bits = 1
- BSRR[16:31] — reset pins when bits = 1 (write-only)

Example register usage:

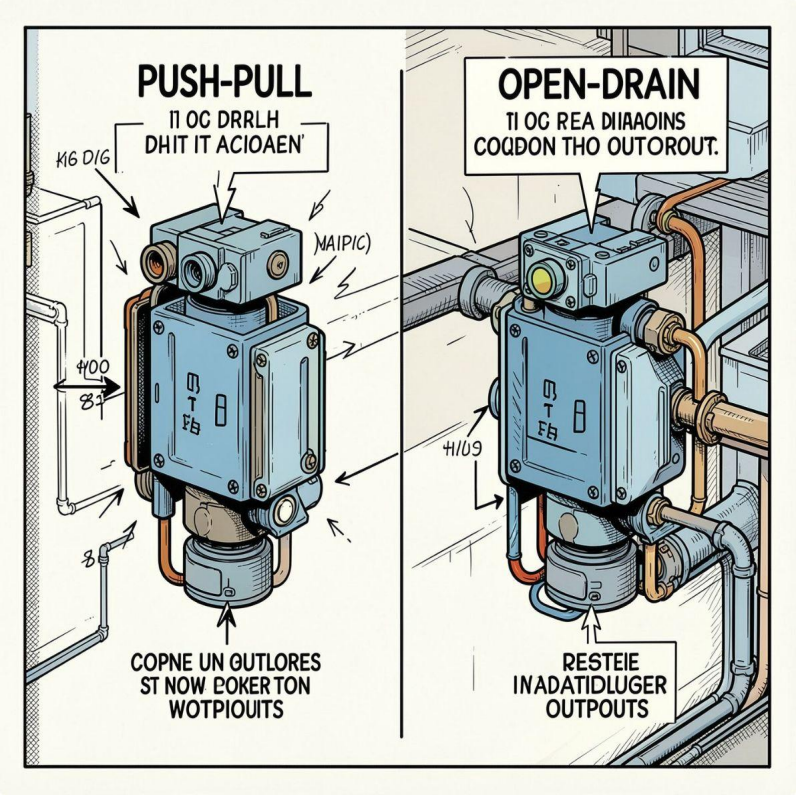
```
GPIOA->BSRR = GPIO_PIN_5;           // set PA5
GPIOA->BSRR = GPIO_PIN_5 << 16;     // reset PA5
```

BSRR avoids read-modify-write races, enabling safe concurrent access from ISRs and multi-master subsystems.

Output Type, Speed, and Electrical Tradeoffs

Output Type

OTYPER: Push-Pull vs Open-Drain



Output Speed — OSPEEDR

OSPEEDR configures rise/fall rates. Higher speed reduces edge time but increases switching current, EMI, and ringing. Principle: use the lowest speed that meets timing and signal-integrity requirements.

- Push-pull: actively drives both HIGH and LOW — good for LEDs, logic signals
- Open-drain: drives LOW only; requires pull-up for HIGH — essential for shared-bus protocols like I²C

Alternate Functions and the AFR Registers

Switching a pin to a peripheral requires two steps:

1. Set MODER to Alternate Function (10)
2. Choose the AF index in AFRL (pins 0–7) or AFRH (pins 8–15); each AF field is 4 bits (AFR[4n+3:4n])

Example: PA5 → set MODER bits for AF, then program AFRL/AFRH to select the specific peripheral function available on PA5 for your package. Always cross-check the STM32F446 datasheet and alternate-function matrix for valid mappings.

Alternate functions are the STM32's pin-routing layer: they connect a physical pad to an on-chip peripheral such as USART, SPI, I²C, timers, or external interrupt lines. Because multiple peripherals may be able to use the same pin, the AFR selection determines which internal signal is actually driven or sampled. This makes pin planning an essential part of board design, especially when a project needs several serial interfaces, PWM outputs, or capture inputs at the same time.

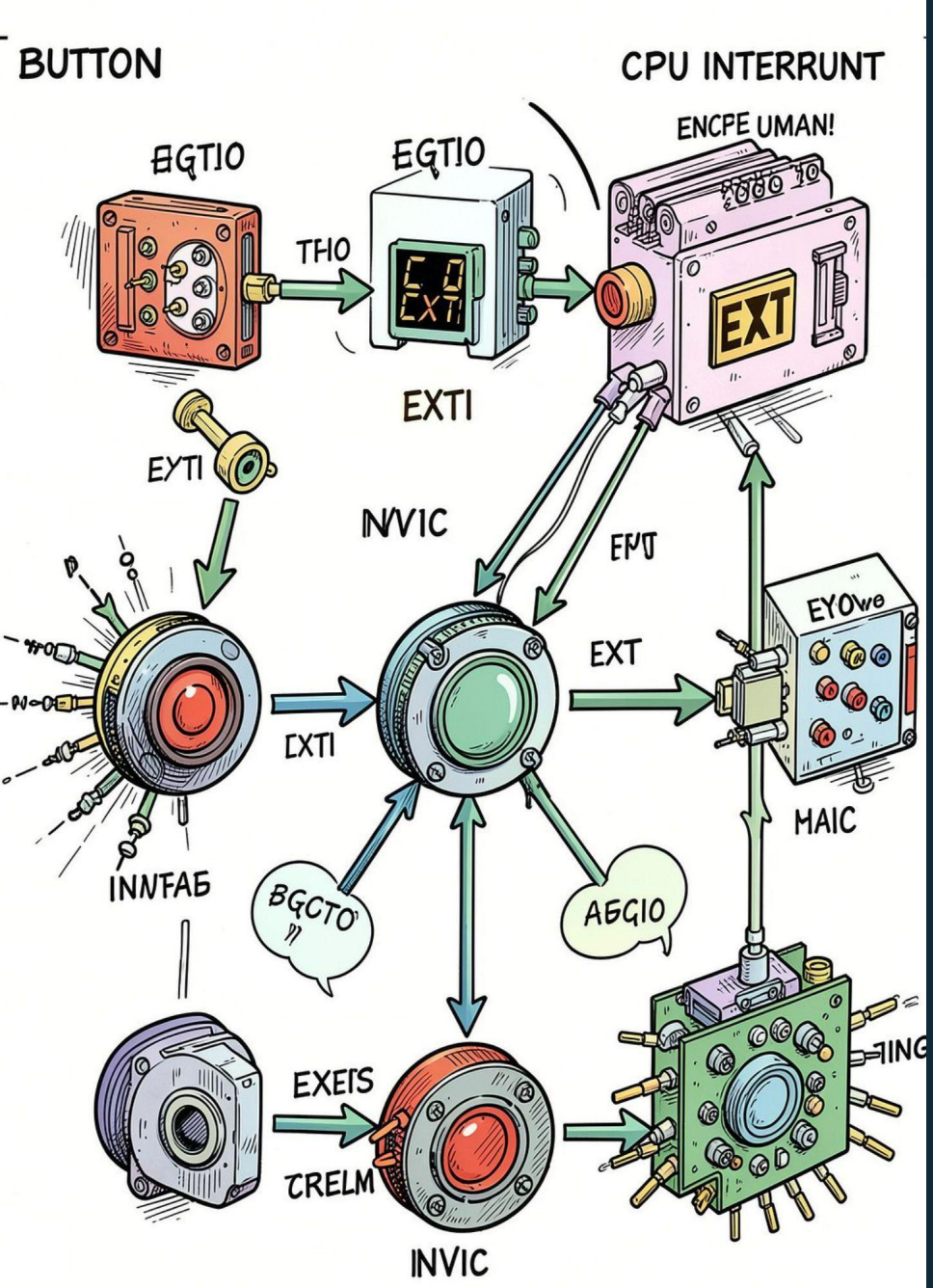
Complete GPIO Configuration Sequence (Register-Level)

Typical initialization for PA5 as a push-pull digital output (register-level):

```
RCC->AHB1ENR |= RCC_AHB1ENR_GPIOAEN;        // enable GPIOA clock
GPIOA->MODER &= ~(3U << (5*2));
GPIOA->MODER |=  (1U << (5*2));             // MODER = 01 (output)
GPIOA->OTYPER &= ~(1U << 5);                // push-pull
GPIOA->OSPEEDR &= ~(3U << (5*2));           // default/low speed
GPIOA->PUPDR &= ~(3U << (5*2));             // no pull
GPIOA->BSRR = GPIO_PIN_5;                   // set PA5
```

HAL abstraction example:

```
HAL_GPIO_WritePin(GPIOA, GPIO_PIN_5, GPIO_PIN_SET);
HAL_GPIO_WritePin(GPIOA, GPIO_PIN_5, GPIO_PIN_RESET);
```



GPIO Interrupts (EXTI) and Practical Interfacing

EXTI connects GPIO lines to the interrupt controller so inputs can trigger CPU interrupts instead of polling. Typical flow: Button press → GPIO input → EXTI triggers → NVIC delivers IRQ → ISR handles event (debounce, state change, toggle LED).

Efficiency

Interrupts eliminate constant polling, reducing CPU load and power consumption.

Responsiveness

Faster reaction to asynchronous events (buttons, sensors, external devices).

Determinism

Atomic BSRR writes plus careful ISR design avoid race conditions in concurrent systems.

Thank You

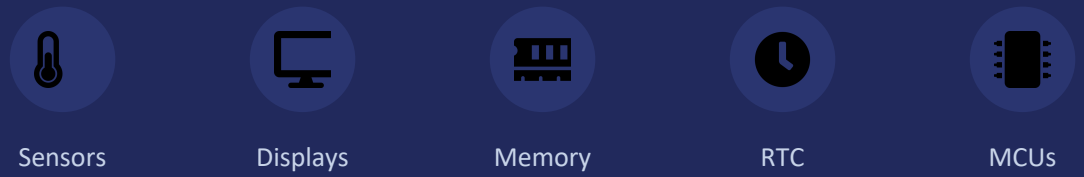
Md.Shoreful islam : 2210876121
MD.Salman Akand : 2110976160
Tusar Talukder : 2211076131

EMBEDDED SYSTEMS • SERIAL COMMUNICATION

I²C Communication Protocol

I²C

Welcome to our presentation



Introduction to Communication Protocol

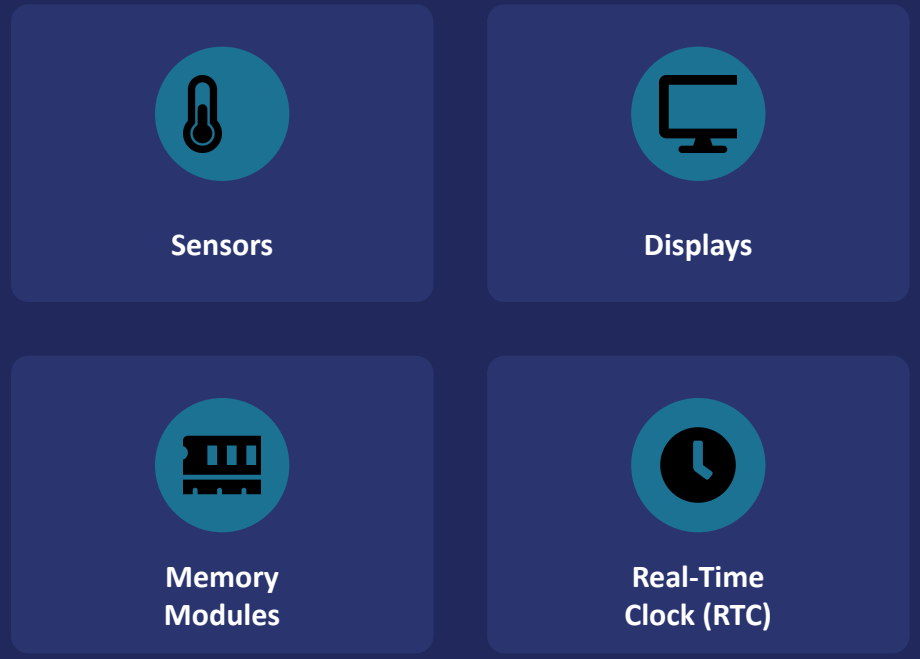
A microcontroller cannot work alone. It needs to communicate with external devices such as sensors, displays, memory modules, and real-time clocks.

To enable this communication, different communication protocols are used:

UART SPI I²C

COMMUNICATION PROTOCOL

A set of rules that allows two or more electronic devices to exchange data reliably.



MCU ↔ Peripheral Communication

Why I²C?

Comparing communication protocols for multi-device systems

Limitations of USART

- Supports only point-to-point communication
- Not suitable for connecting multiple peripheral devices

Limitations of SPI

- Requires multiple communication lines
- Each slave requires a dedicated Chip Select (CS) pin
- Increased wiring complexity for multiple devices

Why I²C?

- Supports multiple devices
- Uses only two wires
- No dedicated Chip Select (CS) line required
- Simple and efficient hardware design

What is I²C?

I²C (Inter-Integrated Circuit) is a synchronous serial communication protocol used for communication between multiple ICs using only two wires.

KEY TERMS

Serial Communication

Data is transmitted bit by bit between devices.

Synchronous Communication

Data transfer is synchronized using a shared clock signal.

Two-Wire Communication

Uses only two lines to connect all devices on the bus:

SDA

Serial Data Line

SCL

Serial Clock Line

SDA carries data; SCL provides the shared timing signal that keeps every device on the bus in sync.

Where is I²C Used?

I²C is widely used for communication between microcontrollers and various peripheral devices.



Display

- OLED Displays



Sensors

- Temperature Sensors
- Pressure Sensors
- Accelerometer & Gyroscope Modules



Memory

- EEPROM

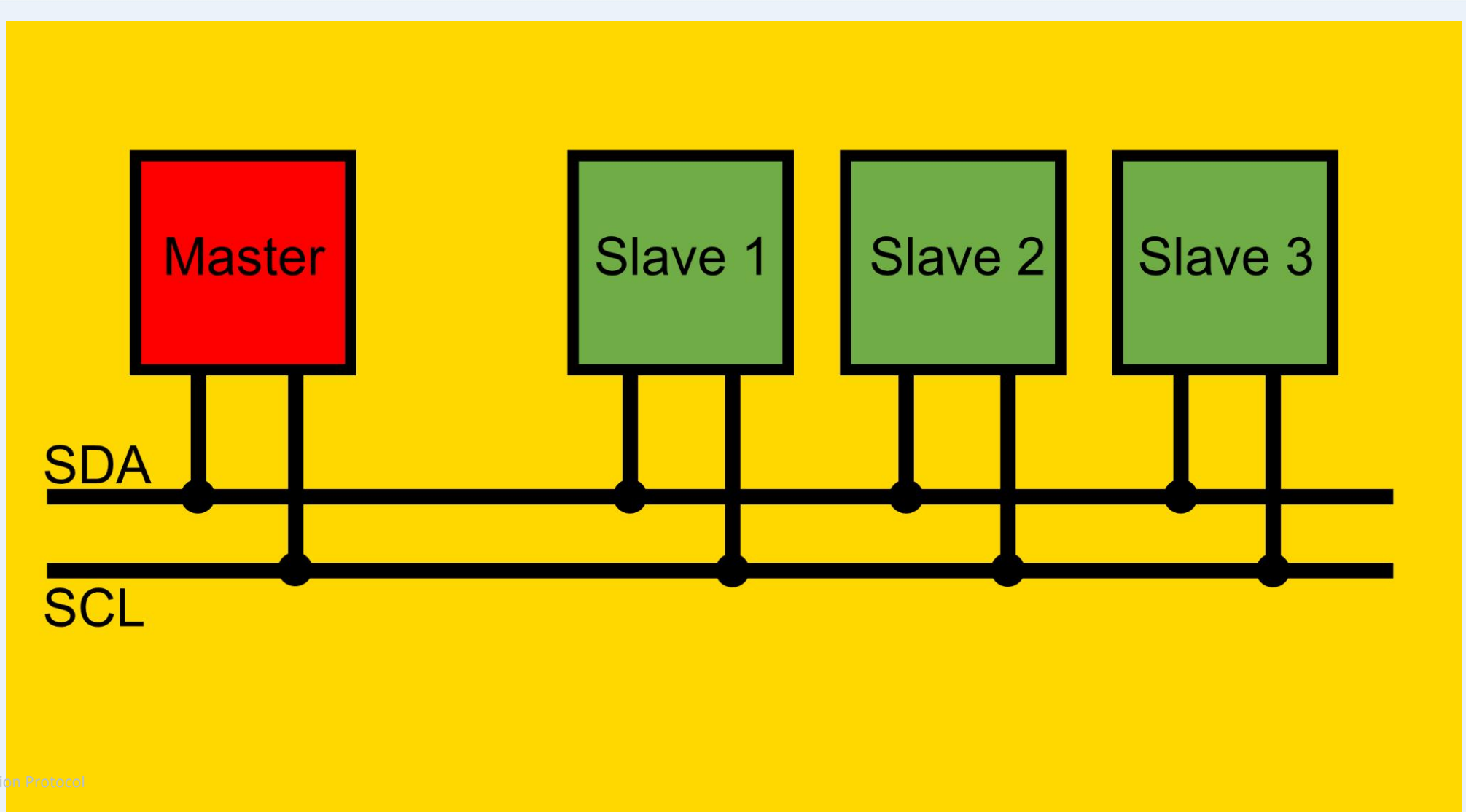


Time Keeping

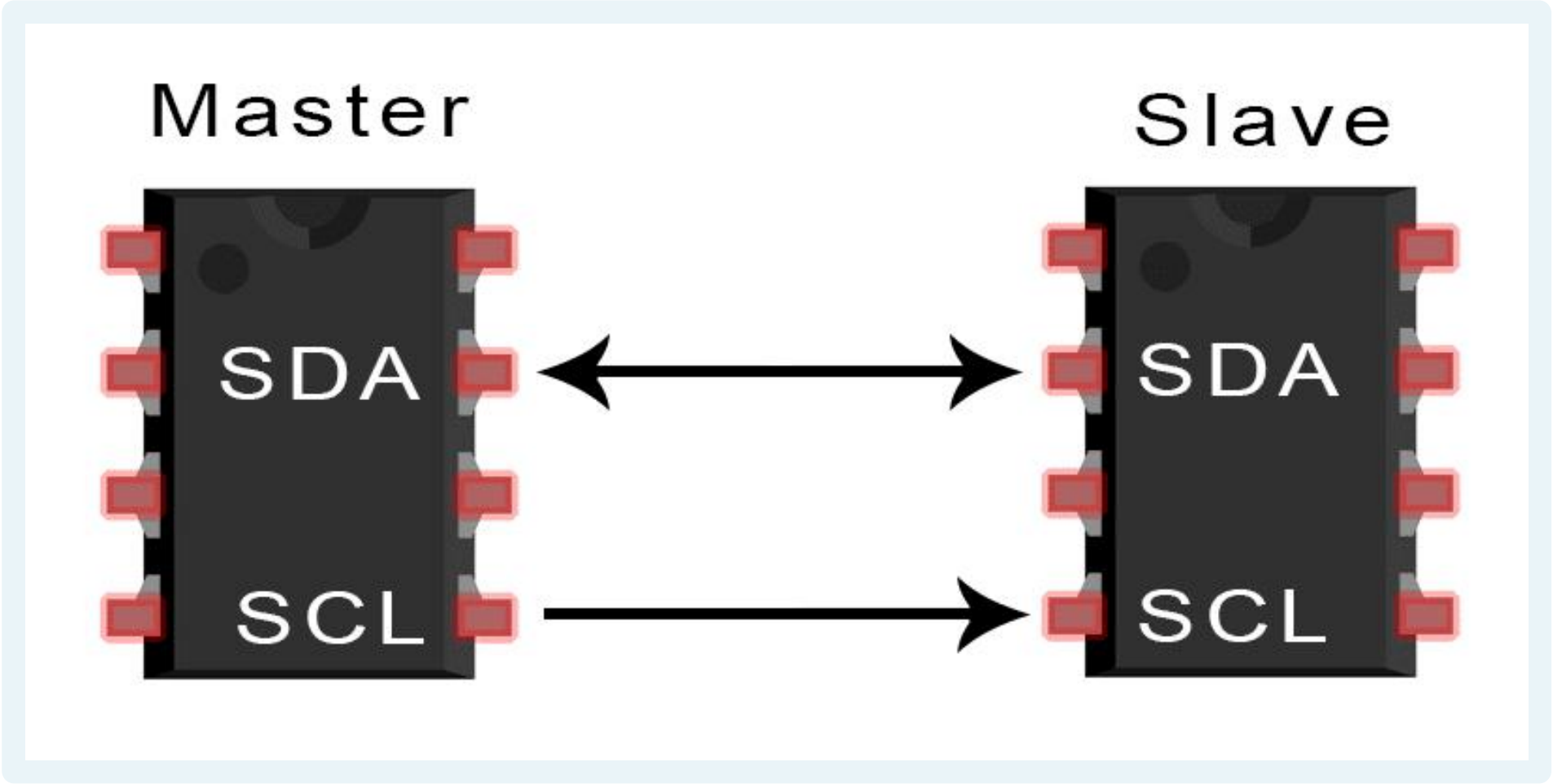
- Real-Time Clock (RTC)

Basic Architecture of I²C

How a master microcontroller connects to multiple slave devices over the I²C bus



I²C Bus Concept



SDA

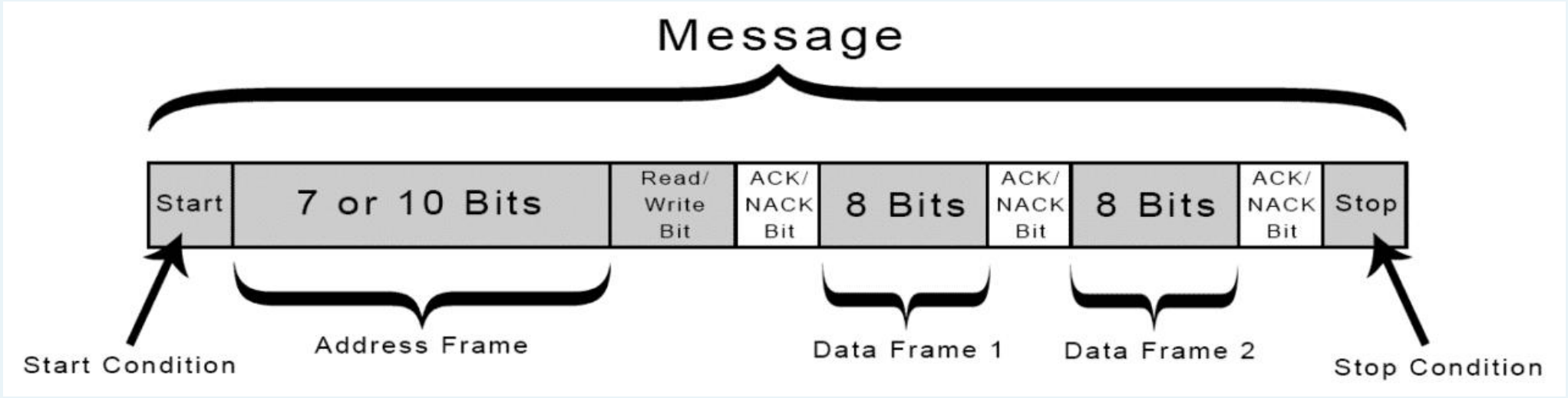
SDA (Serial Data) – The line for the master and slave to send and receive data.

SCL

SCL (Serial Clock) – The line that carries the clock signal.

How I²C Works

I²C transfers data in the form of messages, divided into smaller units called frames.



A MESSAGE CONTAINS

Address Frame → Contains the address of the target slave device

Data Frame(s) → Contains the actual data being transmitted

CONTROL BITS MANAGE COMMUNICATION

Start Condition
Begins communication

Read/Write Bit
Defines data direction

ACK/NACK Bit
Confirms successful reception

Stop Condition
Ends communication

I²C Communication Control Signals

Two signal transitions frame every I²C transaction on the bus

Start Condition

- Indicates the beginning of communication
- SDA changes from HIGH → LOW while SCL remains HIGH

Stop Condition

- Indicates the end of communication
- SDA changes from LOW → HIGH after SCL goes HIGH

I²C Communication Control Bits

Three control fields govern addressing and acknowledgment on the bus



Address Frame

- Contains the 7-bit or 10-bit address of the target slave
- Helps the master select a specific device



Read/Write Bit

- Defines the direction of communication
- 0 → Write data to slave
- 1 → Read data from slave



ACK/NACK Bit

- Confirms whether data is received successfully
- ACK → Successful reception
- NACK → No acknowledgement

I²C Addressing Communication Process

How the master locates and confirms the correct slave device on the bus

1

STEP

Master sends the address of the target slave device

2

STEP

All connected slave devices receive and compare the address

3

STEP

The matching slave sends an ACK signal to the master

4

STEP

Other slaves ignore the message and remain inactive

The Data Frame

How actual data moves across the bus once addressing is complete

- Actual data transfer begins after receiving ACK
- Each data frame contains 8-bit data
- Data is transmitted MSB first
- Receiver sends ACK/NACK after each data frame
- ACK confirms successful data reception
- Master sends STOP condition after completing the transfer

Advantages & Disadvantages of I^2C

Advantages

- Requires only two communication wires (SDA & SCL)
- Supports multiple masters and multiple slave devices
- ACK/NACK mechanism ensures reliable data transfer
- Simplifies hardware design compared to UART-based systems
- Widely used and well-established communication protocol

Disadvantages

- Lower data transfer speed compared to SPI
- Data frame size is limited to 8 bits
- More complex hardware implementation than SPI



Thank You

Thank you all

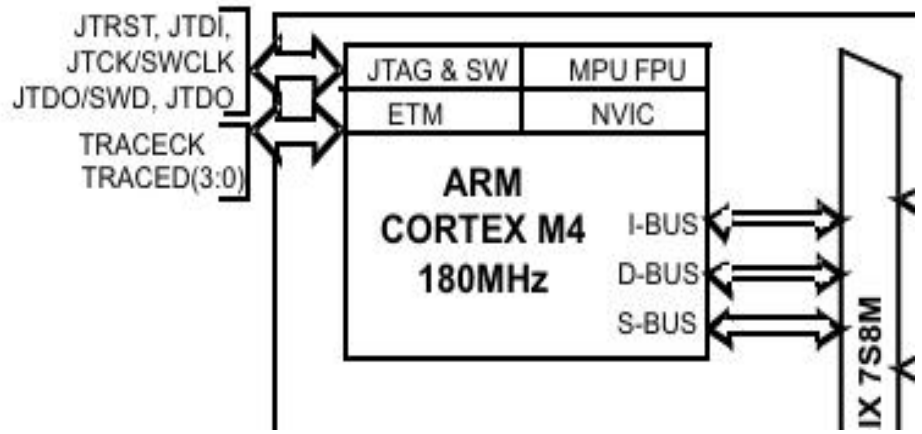
Nested Vectored Interrupt Controller (NVIC)

Group 8:
Tabassum Nishat Aitee – 2212076113
Tasnim Tahmina Jarin – 2212076115
Sadia Afroj Suraya - 2212176122

What is NVIC?

NVIC is a hardware module inside the ARM Cortex-M4 processor which manages all interrupts generated by the microcontroller.

- **It decides which interrupt should be executed first based on priority.**
- **It ensures fast and efficient interrupt handling.**



Why Do We Need NVIC?

Without NVIC:

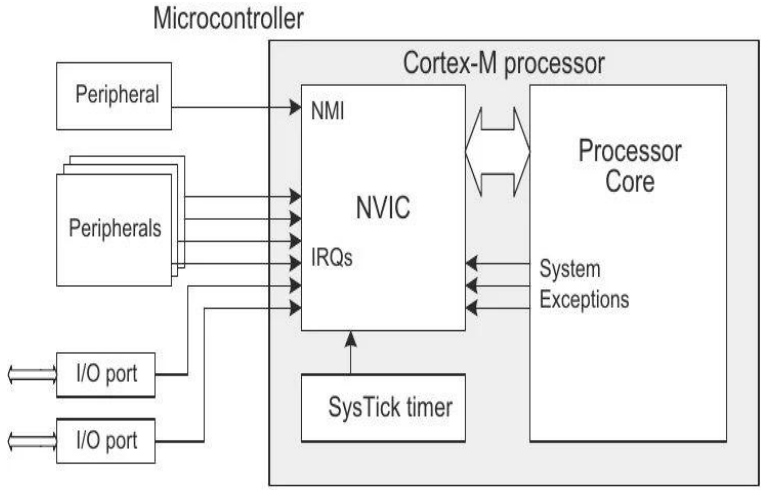
- CPU cannot efficiently manage multiple interrupts.
- Important interrupts may be delayed.

With NVIC:

- Handles multiple interrupt requests.
- Executes higher-priority interrupts first.
- Allows one interrupt to interrupt another if necessary.
- Reduces CPU response time.

How NVIC Handles Interrupts

- **Peripherals** such as Timer, UART, ADC, and GPIO generate **Interrupt Requests (IRQs)**.
- These IRQs are sent to the **NVIC**.
- NVIC **checks, enables, and prioritizes** the interrupt requests.
- The selected interrupt is forwarded to the **Processor Core**.
- The processor executes the corresponding **Interrupt Service Routine (ISR)**.
- After the ISR finishes, the processor **returns to the previous program**.
- **SysTick** and **System Exceptions** can also generate interrupts/exceptions.
- **NMI (Non-Maskable Interrupt)** is used for critical events.



Important Features of NVIC

- Supports **91 maskable interrupt channels**
- Supports **16 Cortex-M4 system exceptions**
- **16 interrupt priority levels**
- Supports **Nested Interrupts**
- Supports **Vectored Interrupts**
- Supports **Tail Chaining**
- Automatic processor state saving/restoring

Interrupt Priority & Nested Interrupts

Interrupt Priority

Each interrupt has a priority.

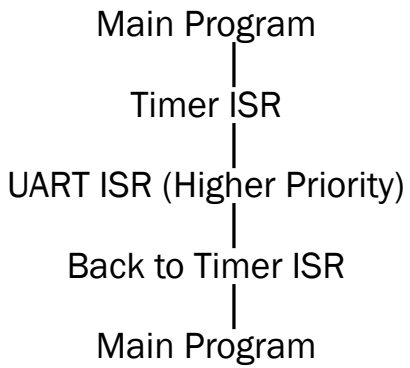
If multiple interrupts occur together:

Higher-priority interrupt executes first.

Interrupt	Priority
HardFault	Highest
Timer	Medium
UART	High

Nested Interrupt

A higher-priority interrupt can interrupt a lower-priority interrupt.



Vectored Interrupt & Interrupt Latency

Vectored Interrupt

Each interrupt has a fixed address in the Interrupt Vector Table.
CPU jumps directly to the correct ISR.

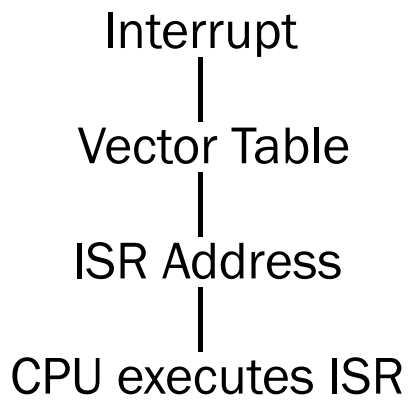
No searching is required.

Interrupt Latency

Time between interrupt occurrence and execution of the ISR.

NVIC reduces latency by:

- Direct vector table access
- Hardware priority checking
- Automatic context saving



Automatic Context Saving & Tail Chaining

Automatic Context Saving

When an interrupt occurs, NVIC automatically saves:

- R0–R3
- R12
- LR
- PC
- xPSR

After ISR execution, these registers are automatically restored.

Tail Chaining

Without Tail Chaining

ISR1

↓

Restore Registers

↓

Save Registers

↓

ISR2

With Tail Chaining

ISR1

↓

ISR2

Advantages

- Fast interrupt response
- Efficient priority management
- Supports nested interrupts
- Reduces CPU workload
- Improves real-time performance
- Faster switching between interrupt

Applications

- Robotics
- Automotive systems
- Industrial automation
- Medical devices
- IoT devices
- Motor control
- Embedded systems

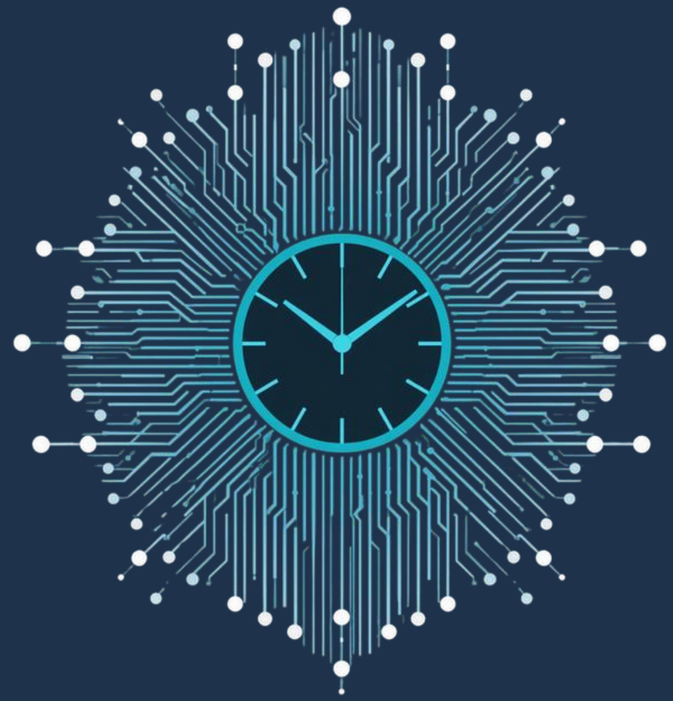


Thank You

EMBEDDED SYSTEMS • FUNDAMENTALS

RTC (Real Time Clock)

Fundamentals and Applications in Embedded Systems



Presentented By:

Shamim Reza- 2110476113

Emon Mia - 2210676120

Raisul Islam - 2111176139

What is a Real Time Clock?

A dedicated hardware clock that continuously tracks real-world time and calendar data—even when the main system is inactive.



01

Accurate time + date

Maintains seconds, minutes, hours and calendar information.

02

Processor independent

Continues running without firmware or CPU intervention.

03

Backup powered

Typically supported by a coin cell or supercapacitor.

Why RTC matters

Time becomes a dependable system resource—not a software assumption.

01

Keeps time when main power is off

02

Time-stamps events and system logs

03

Drives schedules, alarms and timers

04

Fits low-power and battery devices

How RTC works

A stable low-frequency reference is divided, counted and exposed to the host processor.

Crystal oscillator

32.768 kHz

Frequency divider

÷ 2^{15} → 1 Hz

Counter registers

sec • min • hr • date

I²C / SPI

host interface

Backup power

Battery / supercapacitor

The backup domain keeps the oscillator and counters alive while the main rail is absent.

Common RTC ICs

Choose for accuracy, power budget and integration—not familiarity alone.

LOW COST

DS1307

I²C • external crystal
Basic calendar RTC

HIGH ACCURACY

DS3231

TCXO • ±2 ppm class
Temperature compensated

LOW POWER

PCF8563

I²C • low standby current
Portable-device favorite

Applications of RTC

Wherever events need context, RTC provides the when.



01

Embedded systems

Industrial controllers and appliances

02

IoT devices

Smart sensors and wearables

03

Computers + servers

System clock and BIOS

04

Consumer electronics

Cameras and DVRs

Advantages & limitations

ADVANTAGES

01 CPU-independent timekeeping

02 Low power with backup supply

03 Simple I²C / SPI interface

04 Long operational life

LIMITATIONS

01 Crystal quality affects accuracy

02 Backup battery ages over time

03 Clock / calendar scope only

04 Drift without compensation

Reliable time. Independent power.

RTC is a critical component for reliable timekeeping in embedded systems and beyond.



Questions?

Serial Peripheral Interface (SPI) in STM32F446RE

Presented by:
Sumaiya Khatun (2212576141)
Most. Rima Khatun (2112376130)
Most. Faria Jabin Khanam (2212576117)

WHAT IS SPI?

SPI(Serial Peripheral Interface):

SPI is a serial communication protocol used to exchange data between a microcontroller and external devices. It operates using a Master–Slave architecture, where the Master controls the communication by generating the clock signal, and the Slave responds to the Master's commands.

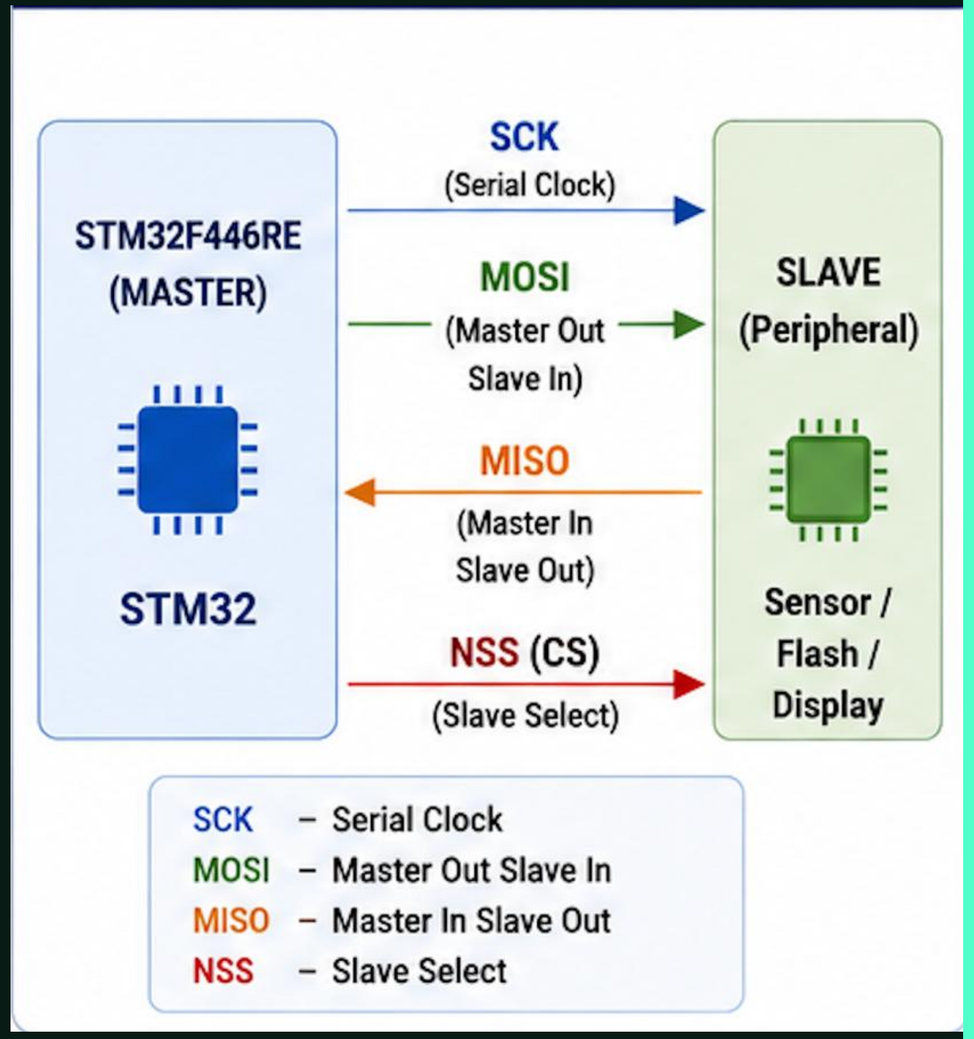
SPI supports full-duplex communication, meaning that both devices can transmit and receive data simultaneously. This makes SPI a fast, simple, and efficient communication protocol, especially for high-speed embedded applications.

Features

- 4 SPI peripherals (SPI1, SPI2, SPI3, SPI4)
- Supports Master and Slave modes
- Full-duplex, Half-duplex, and Simplex communication
- Synchronous communication (provide clock with data)
- 8-bit or 16-bit data frame format
- Programmable baud rate (clock speed)
- Hardware CRC for error detection
- DMA support for high-speed data transfer
- Interrupt support for efficient communication

Communication Lines:

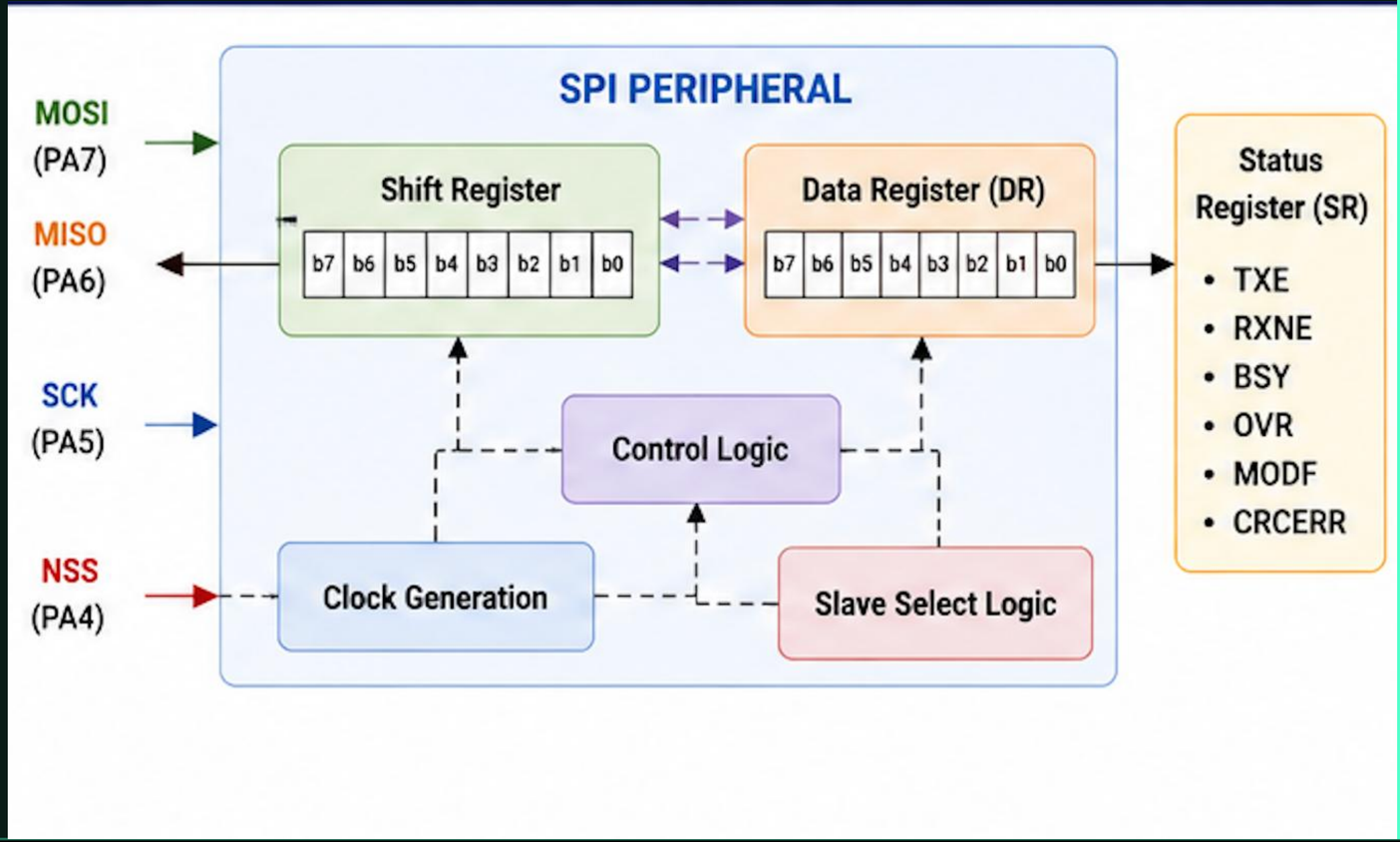
Signals	Function
SCK	Serial Clock generated by Master
MOSI	Master sends data to Slave
MISO	Slave sends data to Master
NSS(CS)	Selects the Slave device



SPI REGISTERS

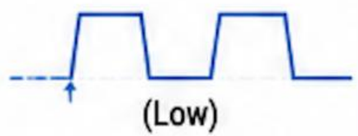
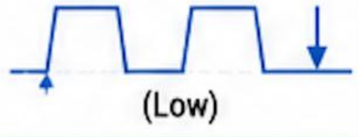
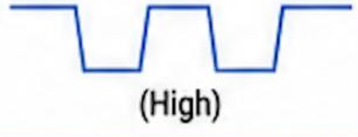
Key Registers

1. **SPI_CR1** is the main configuration register. It sets the operating mode, baud rate, clock polarity, clock phase, and enables the SPI.
2. **SPI_CR2** contains additional configuration settings, such as interrupt and DMA options.
3. **SPI_SR** is the status register. It indicates whether the transmitter or receiver is ready and whether SPI is busy.
4. **SPI_DR** is the data register used to send and receive data.



SPI Clock Modes

- CPOL – Clock Polarity
- CPHA – Clock Phase
- Four modes: Mode 0, 1, 2, 3
- Master and Slave must use the same mode

Mode	CPOL	CPHA	Clock Idle State	Sampling Edge
0	0	0	 (Low)	First (Leading) Edge
1	0	1	 (Low)	Second (Trailing) Edge
2	1	0	 (High)	First (Leading) Edge
3	1	1	 (High)	Second (Trailing) Edge

* Master and Slave must use the same CPOL, CPHA (Mode).

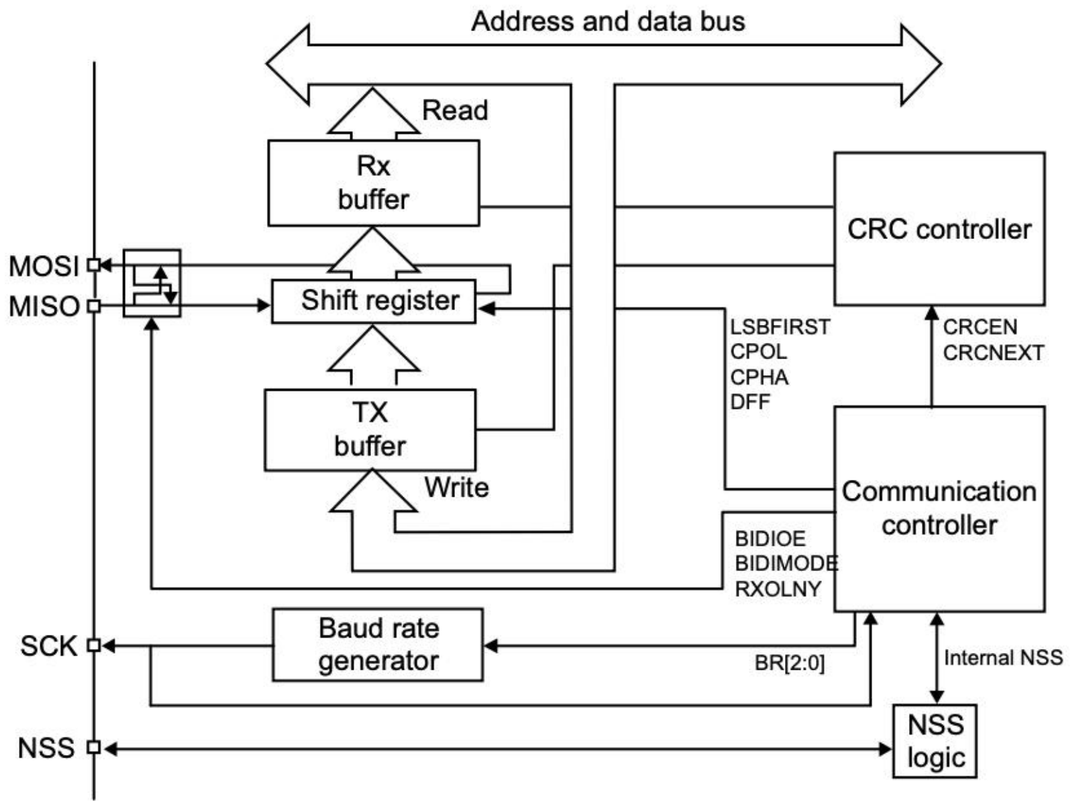
HOW SPI WORKS

Steps in Communication Process

1. Master pulls **NSS LOW**.
2. Master generates **clock (SCK).**
3. Data is sent through **MOSI.**
4. Slave sends data through **MISO.**
5. After transmission, Master sets **NSS HIGH.**

Key Point: SPI transmits and receives data at the same time **(Full-Duplex)**

Figure 303. SPI block diagram

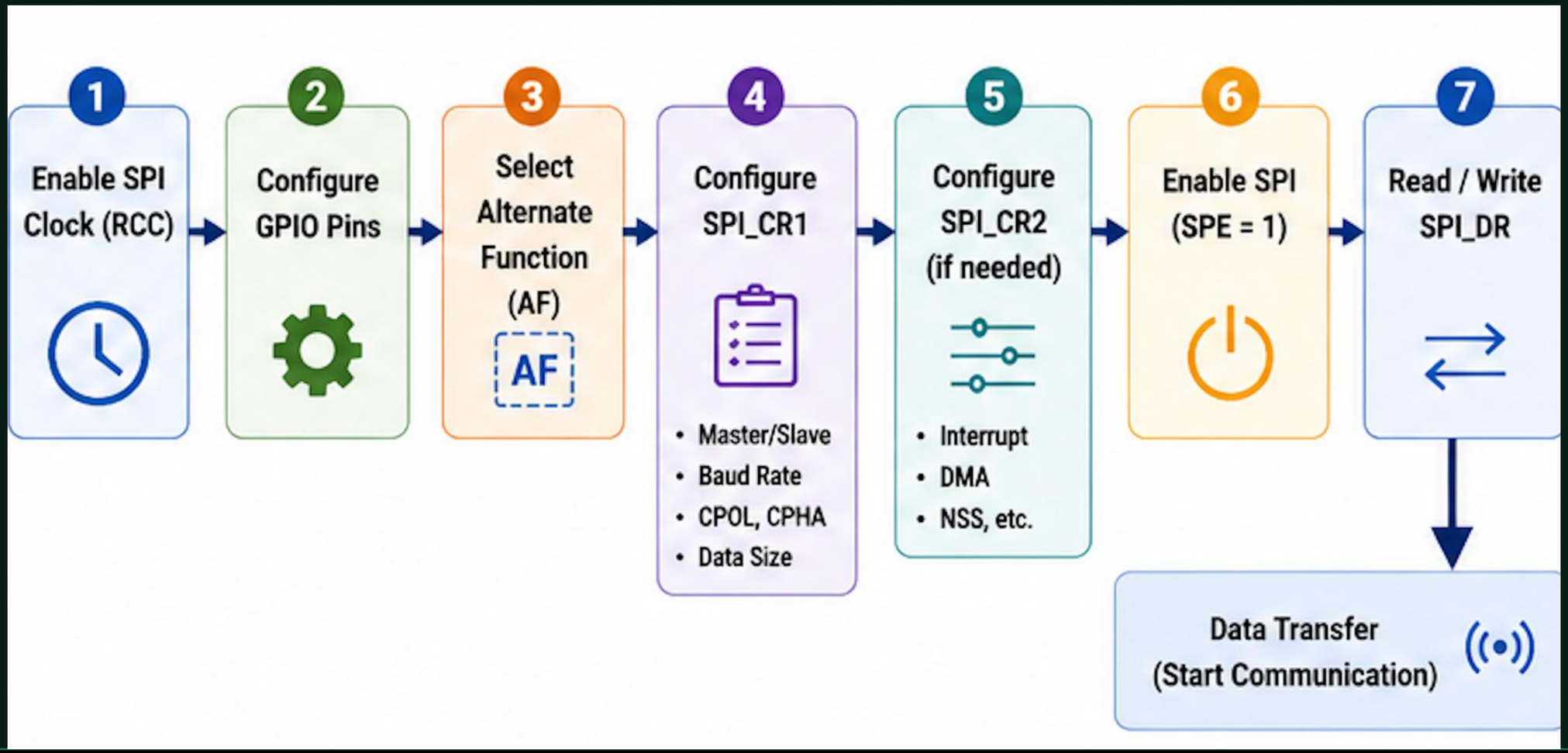


SPI Initialization–(Bare Metal)

The basic steps are:

- Enable the SPI peripheral clock.
- Configure the GPIO pins for Alternate Function mode.
- Configure SPI settings such as Master mode, baud rate, and clock mode.
- Enable SPI by setting the SPE bit.
- Write data to the SPI_DR register to begin communication.

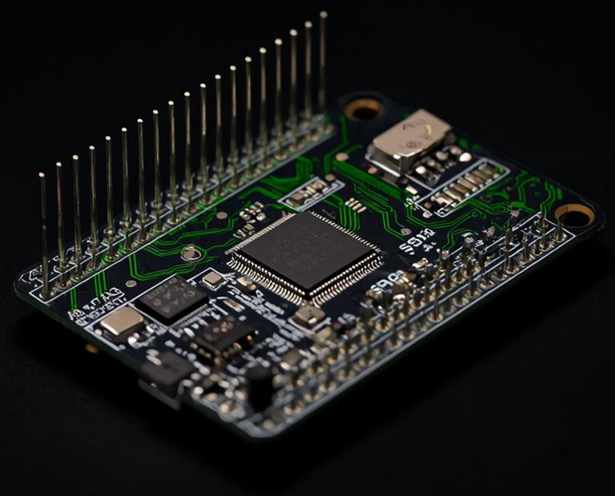
SPI Initialization Flowchart



PIN CONFIGURATION

Default SPI1 Pin Mapping

Signal	Pin
SCK	PA5
MOSI	PA6
MISO	PA7
NSS(CS)	PA4



Applications

SPI Applications

- Sensors — temperature, pressure, motion, etc.
- LCD / TFT Displays — sending display data
- SD Cards — reading and writing data
- Flash Memory — storing and retrieving data
- ADC / DAC — digital and analog data conversion
- Wireless Modules — communication with external modules
- EEPROM / Memory Devices — non-volatile data storage

Key Point: SPI is mainly used for fast and efficient communication between the STM32F446RE and external peripheral devices.

Advantages:

- High-speed data communication
- Full-duplex transfer (simultaneous send & receive)
- Simple hardware interface with few lines
- DMA support for efficient data handling

Disadvantages:

- Requires more wires than I²C
- No built-in device addressing (each slave needs its own NSS/CS line)
- Less suitable when many slave devices are connected

THANK YOU

TIMER

Presented by

MD. NAIMUR RAHMAN — 2111076144
DISHAN IFTE NABIL — 2111076132
A.S.M SABBIR AHMED — 2110676154

What is a Timer?

A hardware peripheral for accurate time control

- A timer is a hardware module inside a microcontroller used to measure and control time.
- It counts clock pulses to create accurate time intervals.
- Timers can generate delays, periodic events, and interrupts.
- Example: blinking an LED every 1 second.

Timer vs Counter

Both count, but their input source is different

TIMER
- Uses the microcontroller’s internal clock.
- Example: generate a 1-second delay.

COUNTER
- Counts external events or pulses.
- Example: count how many times a button is pressed.

Basic Working Principle

How a timer produces a time event

CLOCK
↓
PRESCALER (PSC)
↓
COUNTER (CNT)
↓
Auto Reload Register (ARR)
↓
OVERFLOW / EVENT

1. Clock

Every timer needs a clock.

STM32 timers receive clock from RCC.

Suppose:

Timer Clock = 84 MHz

That means: 84 million counts every second.

Too fast!

So we reduce it.

2. Prescaler

Prescaler divides the clock.

Example:

Clock = 84 MHz
Prescaler = 84

Then,
84 MHz ÷ 84 = 1 MHz

Now,
Timer counts 1 million times per second.

3. Counter (CNT)

Counter stores current value.

0
1
2
3
4
5
...
Each clock pulse
Counter++

4. Auto Reload Register (ARR)

ARR tells the timer:

"When should I restart?"

Example:

ARR = 999

Counter
0
1
2
...
998
999

Next pulse
↓
Overflow
↓
Counter becomes 0 again.

The timer counter frequency is:

f_timer = f_clock / (PSC + 1)

Example:

Clock = 84 MHz
PSC = 83

84 MHz ÷ (83 + 1)
= 1 MHz

So: Counter increases 1 million times per second.

Numerical Example: 1-Second Timer

Using PSC and ARR to create a 1-second period

Clock = 72 MHz
PSC = 7199
CNT Clock = 10 kHz
ARR = 9999

1 Second → EVENT

Step 1: fCNT = 72,000,000 ÷ (7,199 + 1) = 10,000 Hz

Step 2: Period = (ARR + 1) ÷ fCNT = 10,000 ÷ 10,000 = 1 second

Result: So the timer generates an update event every 1 second.

Application: LED Blinking

A simple real-world timer example

SET (Clock + Prescaler)

↓

START (Start Timer)

↓

WAIT (CNT reaches ARR)

↓

TOGGLE (Toggle LED)

↓

REPEAT

Result: The LED turns ON and OFF at a fixed time interval.

Important Timer Registers

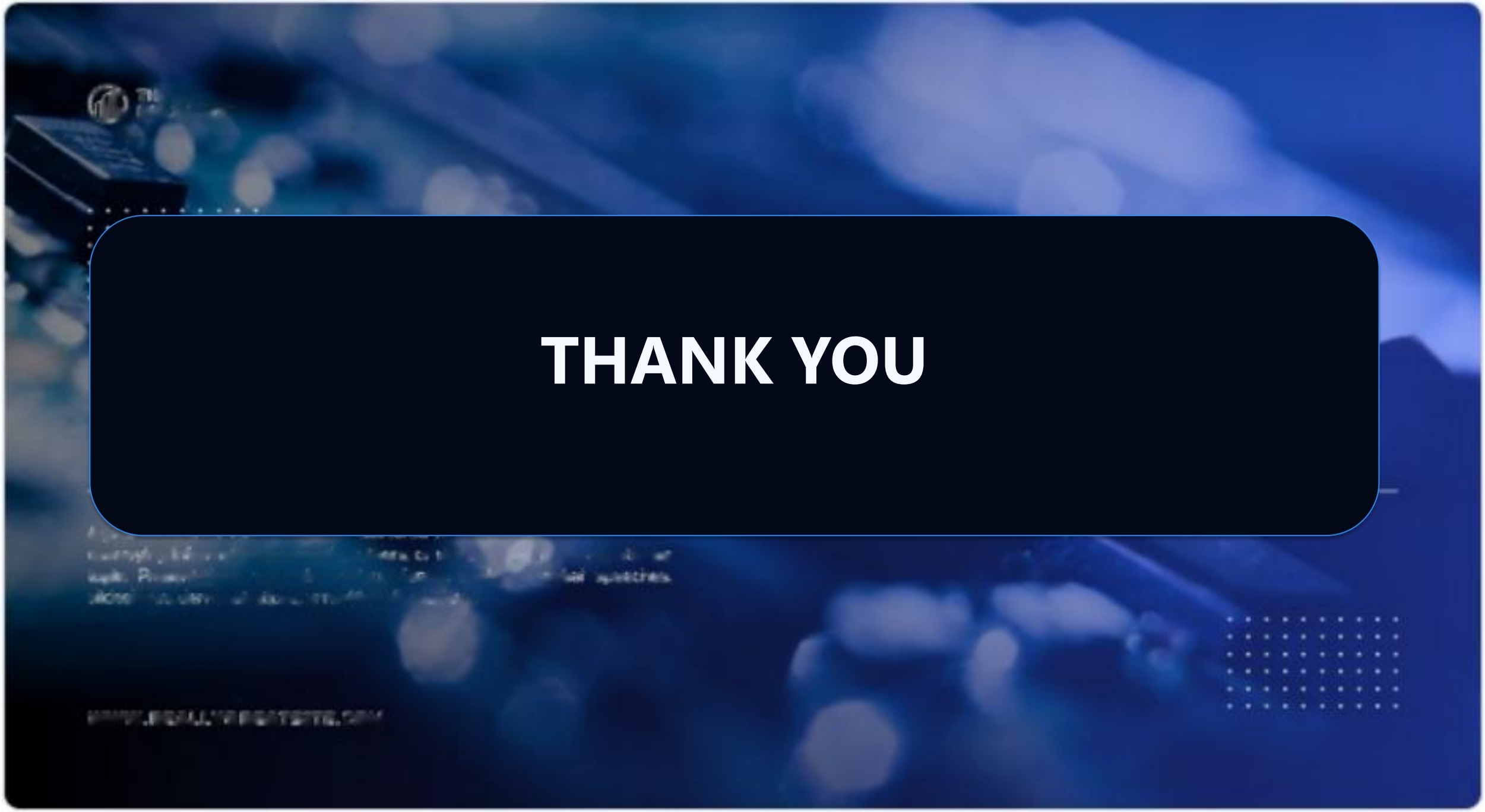
Typical registers used to configure a timer

Prescaler — Divides input clock.
Counter register — Current counter value.
Auto-Reload — Sets counter limit / period.
Control — Starts and configures timer.
Status — Shows timer events.

Conclusion

Why timers matter in embedded systems

- Timer is an essential microcontroller peripheral.
- It measures and controls time accurately.
- It can generate delays, interrupts, periodic signals and PWM.
- Timers are widely used in embedded and real-time applications.


THANK YOU



USART / UART

Presentation

Group 10

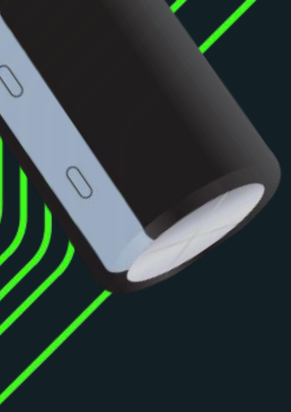
Our Team

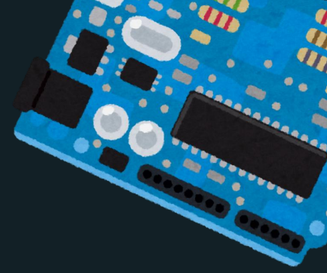
Mahfuzur Rahman
(2110976149)

Washim Akram
(2110276110)

Md. Akibuzzaman
(2210976128)



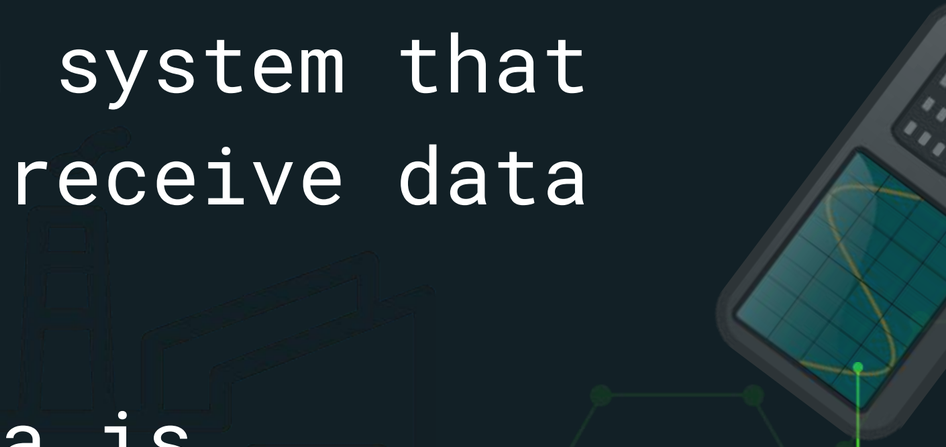


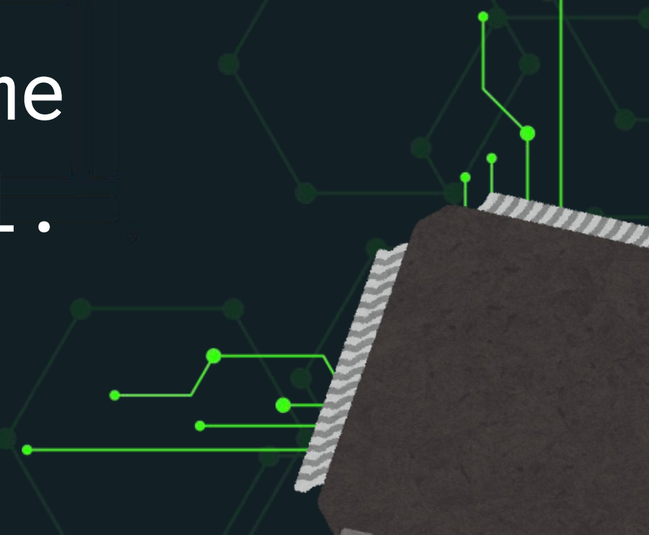


What is UART / USART ?

UART/USART is a communication system that allows the STM32 to send and receive data serially.

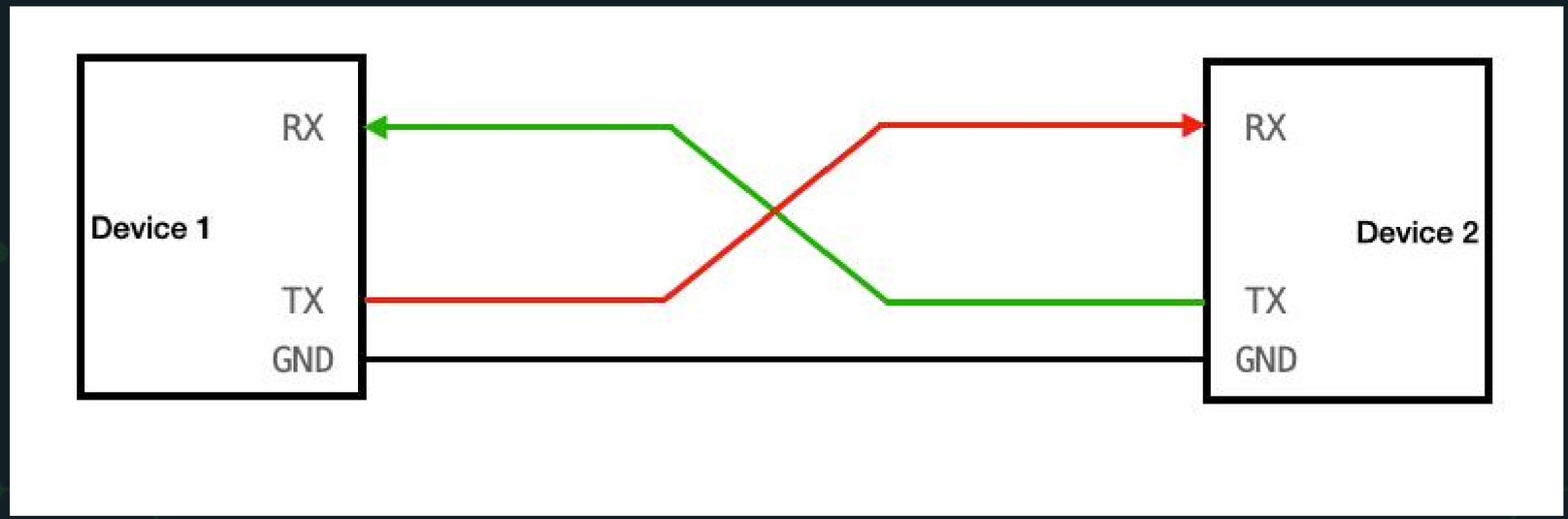
Serially means data is transmitted one bit at a time over a communication channel.





UART = Universal Asynchronous Receiver/Transmitter

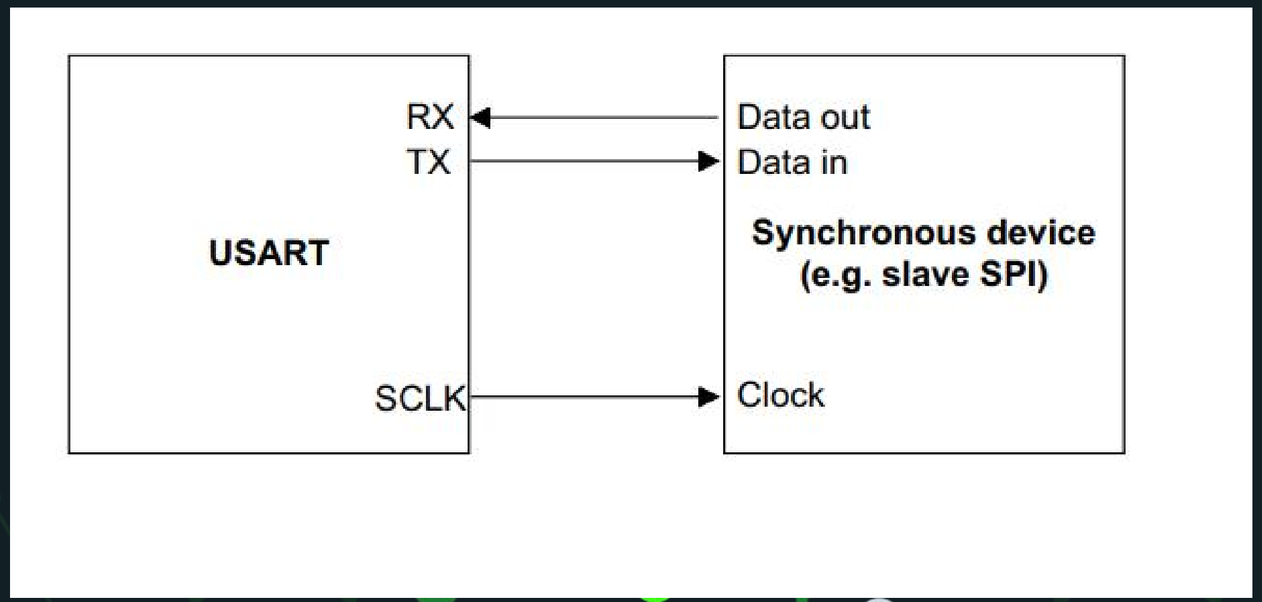
UART is a hardware peripheral/protocol used to transmit and receive serial data without a separate clock signal.



USART = Universal Synchronous/ Asynchronous Receiver/Transmitter

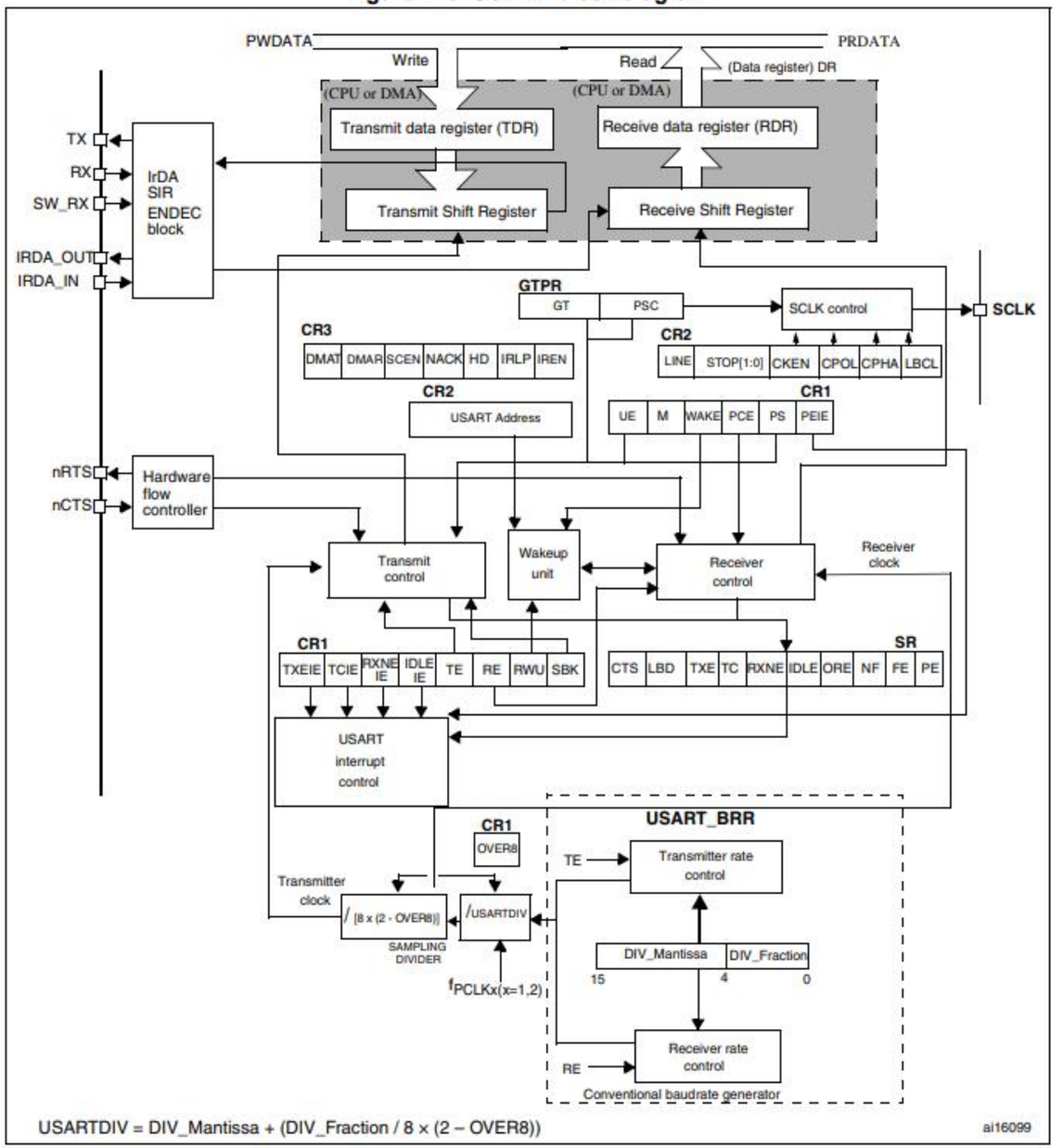
USART can do UART-style asynchronous communication, plus synchronous communication.

This is why the peripheral is called USART



USART Block Diagram

Figure 278. USART block diagram



What are TX and RX?

TX = Transmit
It sends data.

RX = Receive
It receives data.

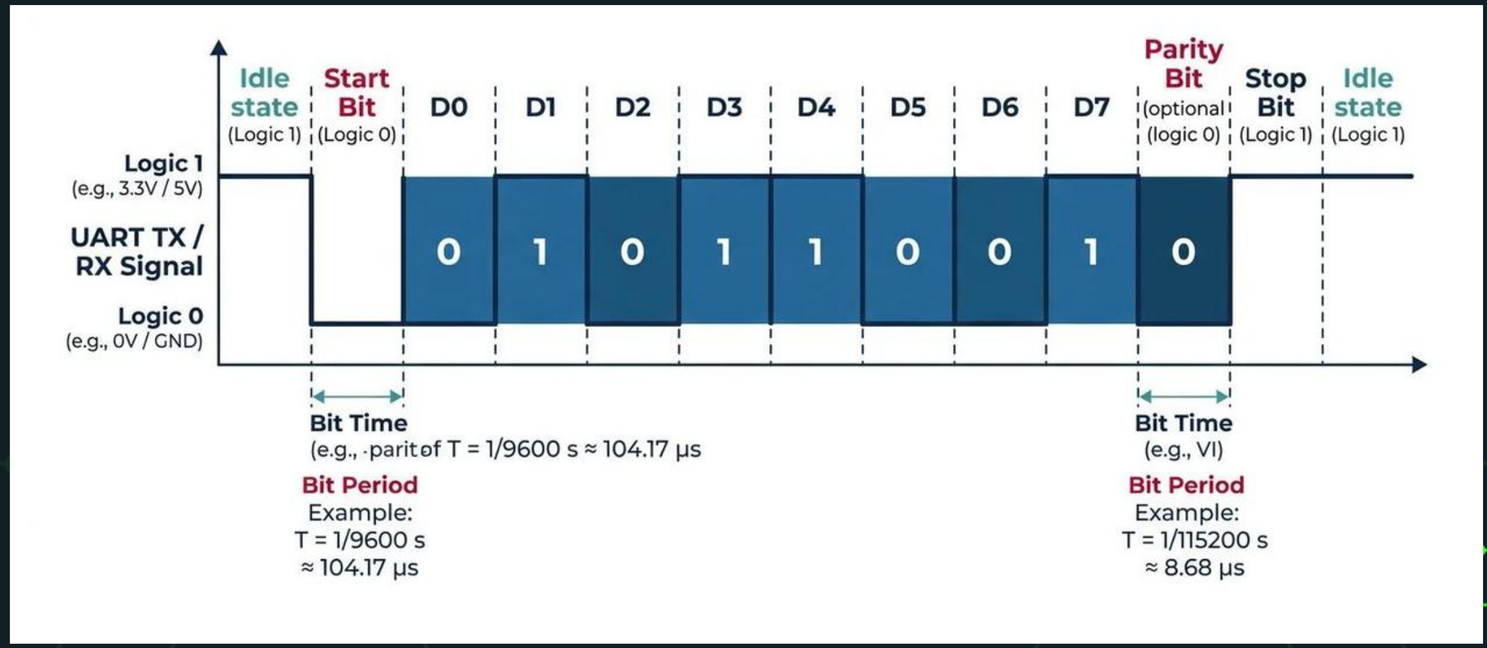
Why do we need GND?

We need GND (ground) in UART because voltage is a relative measurement. Without a shared reference point, the receiving device cannot distinguish between high logic (e.g., 3.3V or 5V) and low logic (0V)

UART Frame Structure

A UART frame consists of:

- Start Bit
- Stop Bit
- Data Bits
- Parity Bit(optional)



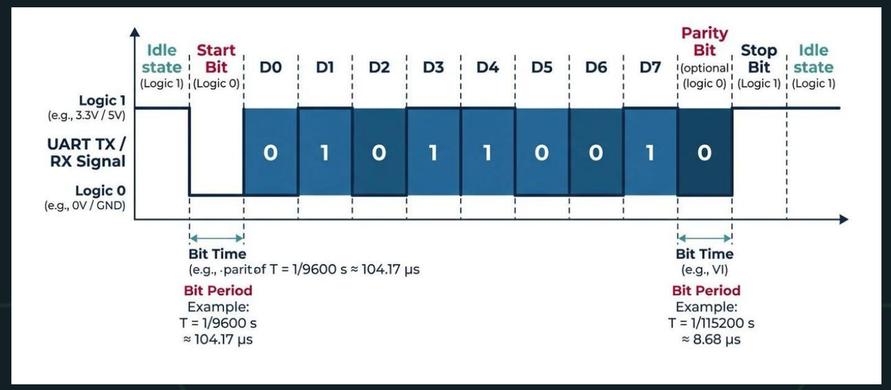
Start Bit:
Tells the receiver a new frame is starting

Data Bits:
Usually 5–9 bits
but most common is 8 bit

Parity:
Used for simple error detection.

- Even parity
- Odd parity
- No parity

Stop Bit:
Indicates the end of the frame.
1 stop bit or 2 stop bits



Example: Sending the Character 'A'

ASCII: A = 65

Binary:01000001

For an 8N1 UART frame:

START	DATA	STOP
0	01000001	1
|	|	|
Start	8 bits	Stop

UART normally transmits the data bits LSB first.

1 0 0 0 0 0 1 0

Baud Rate

Baud rate represents the number of symbols transmitted per second

Common values:

- 4800
- 9600
- 19200
- 38400
- 57600
- 115200

Example:Baud Rate = 9600

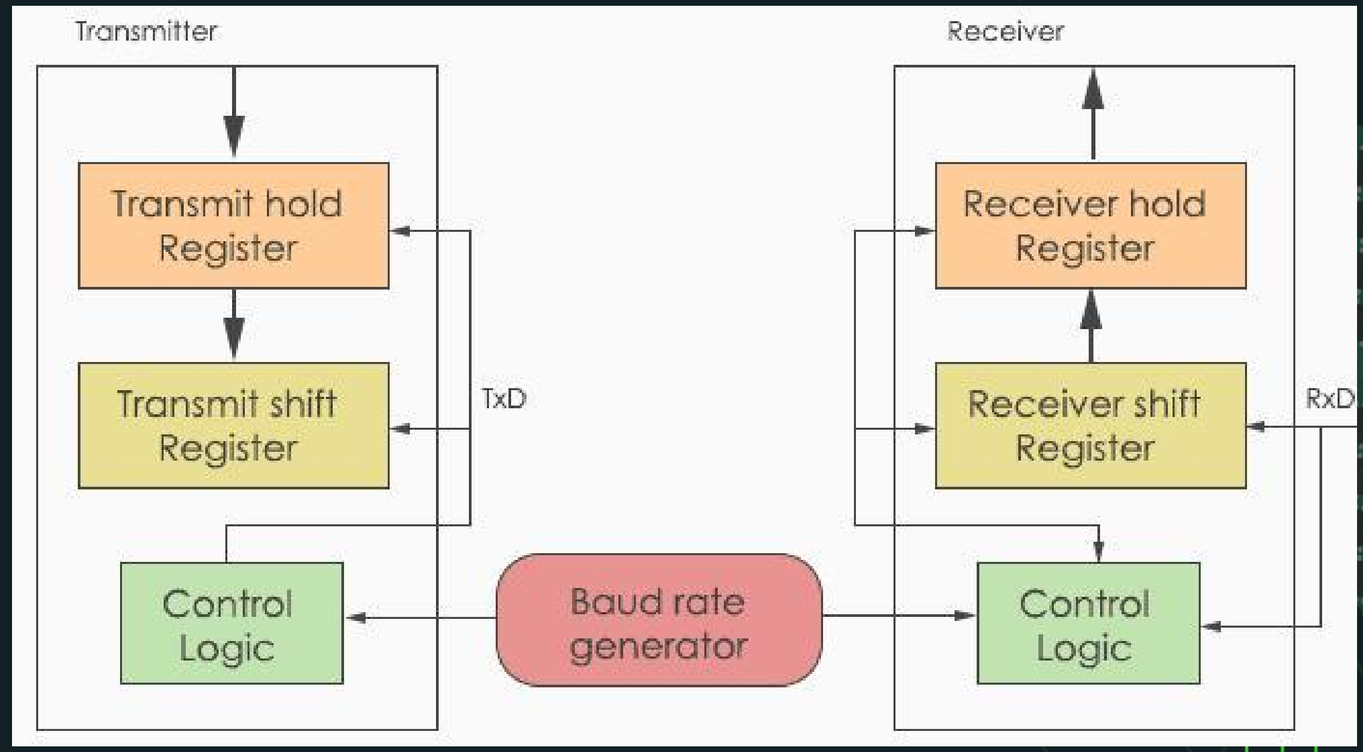
So approximately 1 bit = 1/9600

= 104.17 µs

USART Data Transmission and Reception

For transmission, the CPU puts data into the Transmit Data Register. The USART moves the data to the transmit shift register, converts it into a serial frame, and sends it through the TX pin.

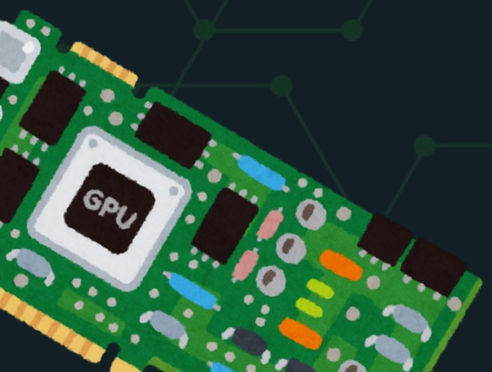
For reception, serial data arrives through the RX pin. The USART receives and reconstructs the frame, stores the data in the Receive Data Register, and the CPU can then read it.

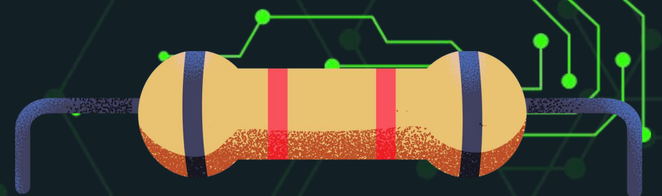


USART Registers

Register	Purpose
USART_SR	Status information
USART_DR	Data transmission/reception
USART_BRR	Baud-rate configuration
USART_CR1	Main control
USART_CR2	Additional control
USART_CR3	DMA/flow-control/other features
USART_GTPR	Guard time & prescaler












Thank
You

USART

Universal Synchronous Asynchronous Receiver Transmitter

Team Members

Shuvo Roy — 2210976106

Md Ahsan Habib Sadik — 2210976101

Md. Redoanul Haque Zim — 2210976139

Made with GAMMA

OVERVIEW

Overview & Key Features

USART is a highly flexible serial communication peripheral capable of operating in multiple modes, making it a cornerstone of embedded system communication design.



Full-Duplex NRZ

Simultaneous transmit and receive using standard Non-Return-to-Zero asynchronous serial format.



Multi-Mode Support

Operates in synchronous, half-duplex, and multiprocessor networking configurations.



Protocol Compatibility

Compatible with LIN, Smartcard (ISO 7816), IrDA SIR ENDEC, and DMA-based data transfer.

Made with GAMMA

HARDWARE

Essential Pins

USART relies on a defined set of signal pins to manage data flow, clocking, and hardware handshaking. Each pin serves a distinct role in maintaining reliable communication.

SCLK — Clock Output

Provides a clock signal to the receiver in synchronous communication mode only.

nCTS — Clear to Send

Hardware flow control input. Pauses transmission when asserted by the receiving device.

nRTS — Request to Send

Hardware flow control output. Signals readiness to receive data from the remote device.

TX — Transmit Data Output

Transmits serial data outward. Held at logic high (idle state) when no transmission is active.

RX — Receive Data Input

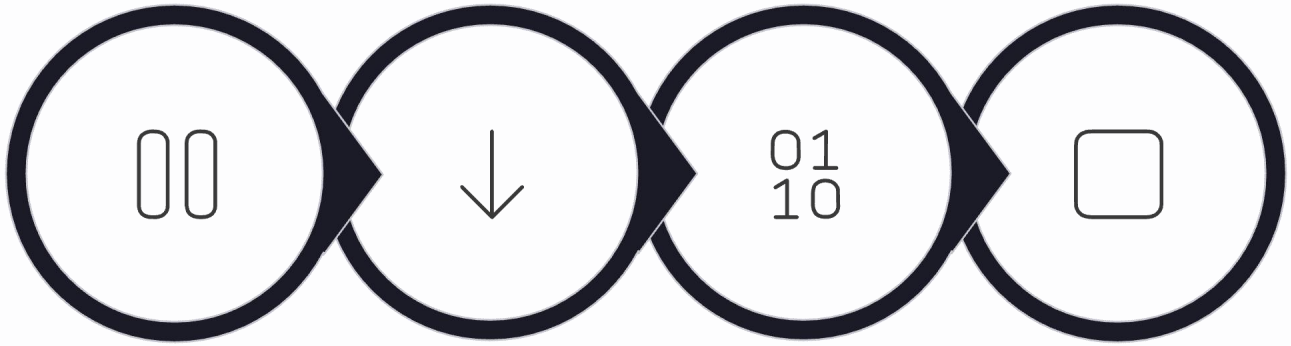
Accepts incoming serial data. Employs oversampling noise filtering to ensure signal integrity.

Made with GAMMA

FRAME STRUCTURE

Character Frame Structure

Every USART transmission follows a defined frame structure. Understanding each field is essential for correct configuration and reliable data exchange.



Idle Line | Start Bit | Data Word | Stop Bits

Idle Line

The line is held at logic "1" before any transmission begins, signaling an inactive but ready state.

Start Bit

A single logic "0" marks the beginning of each character frame, synchronizing the receiver.

Data Word

8 or 9 data bits transmitted LSB (Least Significant Bit) first, configurable via the M bit in USART_CR1.

Stop Bits

Configurable stop duration: 0.5, 1, 1.5, or 2 bits, providing inter-frame separation and synchronization.

Made with GAMMA

TRANSMITTER

Transmitter Operation

The USART transmitter follows a precise register-driven sequence. Each step must be completed in order to ensure correct and complete frame transmission.



Enable TE Bit

Set the **TE** bit in `USART_CR1` to activate the transmitter and prepare the idle line.



Write to DR

Write data to `USART_DR`. It is automatically moved to the Transmit Shift Register (TSR).



Monitor TXE

The **TXE** flag goes high when the TDR is empty and ready to accept the next data byte.



Confirm TC

The **TC** flag is set once the full frame including stop bits has been completely shifted out.

Made with GAMMA

RECEIVER & BAUD RATE

Receiver Operation & Baud Rate

Receiver Operation

The USART receiver continuously monitors the RX line for a valid start bit. Once detected, it samples incoming bits using either **8× or 16× oversampling** to filter noise and reconstruct the data word accurately. Upon successful reception of a complete character frame, the **RXNE** (Read Data Register Not Empty) flag is set, signaling the application to read the received byte from `USART_DR`.

Hardware Flow Control

The **RTS/CTS** mechanism prevents receiver overrun and buffer overflow. RTS signals readiness to receive; CTS pauses the transmitter when the peer is not ready — critical in high-throughput applications.

Baud Rate Configuration

The baud rate is derived from the peripheral clock using fractional division via the `USARTDIV` register.

Fractional baud rate generation minimizes clock error across standard baud rates such as 9600, 115200, and beyond.

12-bit Mantissa

Integer portion of the clock divisor value.

3/4-bit Fraction

Fractional portion enabling precise baud rate tuning.

Made with GAMMA

Thank You

Made with GAMMA

MICROPROCESSOR & MICROCONTROLLER



WATCHDOG TIMER

(WDG)

Functionality, Working Principle & Real-World Use Cases

PRESENTED BY

Md Habibur Rahman	2210376108
Soumithra Debnath Shanto	2210976136
Md Habibur Rahman	2210976107

AGENDA

What We Will Cover

1. Introduction

What a watchdog timer is, and why embedded systems need one

2. Types of Watchdog

Independent (IWDG) vs Window (WWDG) watchdogs

3. How It Works

The core countdown-and-refresh mechanism, register level

4. STM32 Implementation

IWDG & WWDG architecture on the STM32F103C8T6

5. Use Cases

Automotive, industrial, IoT, medical & aerospace applications

6. Pros, Cons & Summary

Advantages, limitations and best practices

INTRODUCTION

What Is a Watchdog Timer?

FULL FORM

WDG = Watchdog Timer

A hardware timing circuit built into the microcontroller that monitors the health of running software.

Definition

A Watchdog Timer (WDG) is a free-running hardware counter that automatically resets the microcontroller if the application software fails to periodically restart ("refresh" or "kick") it in time — for example, when the CPU is stuck in an infinite loop, a deadlock, or a corrupted execution path.

Why It Is Needed

Embedded systems often run unattended for months or years — a car ECU, a remote sensor, a factory controller. They cannot wait for a human to notice a freeze and press reset. The watchdog gives the system the ability to detect its own malfunction and recover automatically, without any manual intervention.



The Guard-Dog Analogy

Think of the watchdog as a guard dog that must be "petted" (refreshed) regularly. As long as you keep petting it on schedule, it stays calm. The moment you stop — because you are asleep, stuck, or unable to respond — it "barks": it resets the system.

"Pet on time → system runs.
Miss the pet → system resets."

CLASSIFICATION

Two Types of Watchdog Timers



IWDG

Independent Watchdog

- Runs on its own separate low-speed clock (LSI, ~40 kHz on STM32)
- Fully independent of the main system clock
- Keeps working even if the main oscillator fails or is misconfigured
- Simple: must be refreshed before a single timeout deadline
- Best for detecting total software lock-ups / hangs



WWDG

Window Watchdog

- Runs on the main peripheral clock (APB1) with a prescaler
- Must be refreshed inside a specific time "window"
- Refreshing too early OR too late both trigger a reset
- Catches both slow tasks and abnormally fast / skipped tasks
- Best for detecting timing & sequencing faults

HOW IT WORKS

The Core Countdown-and-Refresh Mechanism



A timeout value is loaded into the watchdog's down-counter register at startup.

The counter decreases continuously, driven by its own clock — independent of the CPU's main program.

The application must periodically write a specific "refresh" value to reload the counter before it hits zero.

If the refresh never arrives in time — because code is hung — the counter reaches zero.

A reset signal is generated automatically, restarting the microcontroller from its boot vector.

Key Idea: The watchdog runs independently of the main program. If the CPU is "alive" but stuck in a bad loop, only an independent timer can detect it — the frozen code cannot detect its own freeze.

STM32 IMPLEMENTATION

Independent Watchdog (IWDG) — Register View



Key Registers

`IWDG_KR`	Key register — write 0xCCCC to start, 0xAAAA to refresh, 0x5555 to unlock config
`IWDG_PR`	Prescaler register — sets the clock divider (÷4 to ÷256)
`IWDG_RLR`	Reload register (12-bit) — sets the timeout / down-count start value
`IWDG_SR`	Status register — flags when PR/RLR updates are still in progress

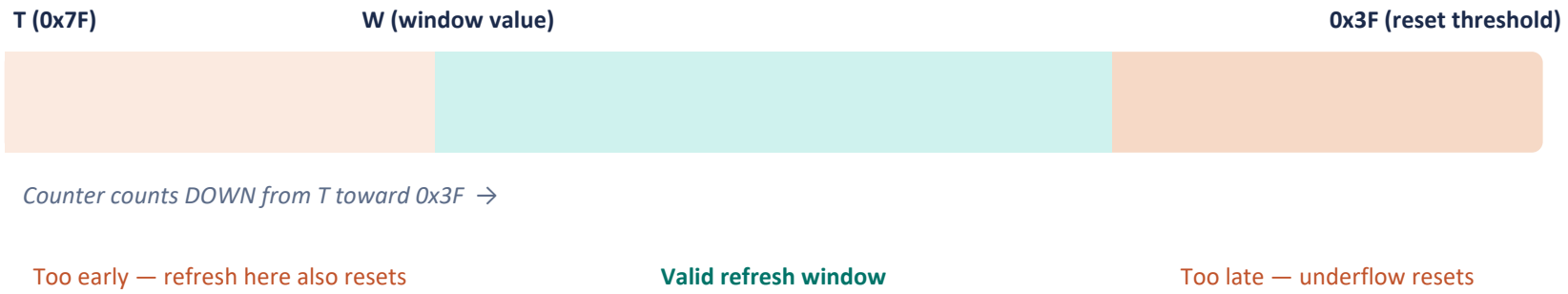
EXAMPLE — refresh call inside main loop

```
IWDG->KR = 0x5555;      // unlock PR & RLR
IWDG->PR  = 0x04;        // set prescaler
IWDG->RLR = 0x0FFF;     // set reload / timeout
IWDG->KR = 0xCCCC;      // start the watchdog
while (1) {
   run_application_tasks();
   IWDG->KR = 0xAAAA;   // refresh (kick)
}
```

STM32 IMPLEMENTATION

Window Watchdog (WWDG)

Unlike the IWDG, the WWDG defines a valid **window** of time to refresh in — bounded by an upper limit (T) and a lower limit (W). Refreshing outside this window, in either direction, triggers a reset.



Why a window at all?

A single-timeout watchdog (like IWDG) only catches code that is too slow. The WWDG additionally catches code that runs abnormally fast — e.g. a task that skips steps due to memory corruption or a jump to the wrong address. This makes it well suited for safety-critical timing supervision, such as verifying a control loop executes at a precise, expected rate.

Key Registers

`WWDG_CR`

7-bit down-counter T[6:0] + watchdog enable bit

`WWDG_CFR`

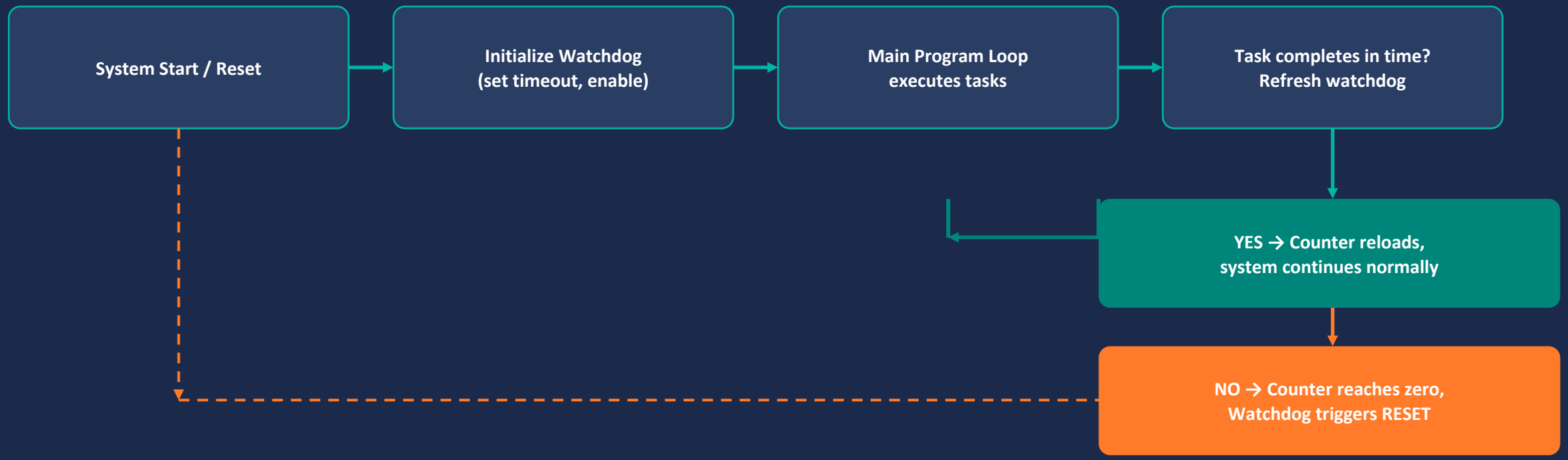
Window value W[6:0] + clock prescaler (WDGTB)

`WWDG_SR`

Early-wakeup interrupt flag (EWIF)

PROCESS FLOW

Watchdog Operation — End to End



Dashed line: a reset restarts execution from the boot vector, re-entering the flow at "System Start."

APPLICATIONS

Where Watchdog Timers Are Used



Automotive ECUs

Engine, braking & airbag control units use watchdogs to guarantee fail-safe recovery in safety-critical systems.



Industrial Automation

PLCs and factory controllers run unattended for years; a watchdog keeps production lines from silently freezing.



IoT & Remote Sensors

Field-deployed nodes cannot be manually rebooted — the watchdog is the only recovery mechanism available.



Medical Devices

Patient monitors and infusion pumps require guaranteed responsiveness; a hang must self-correct immediately.



Consumer Electronics

Routers, smart appliances & set-top boxes silently recover from firmware glitches without user action.



Aerospace Systems

Satellites and autonomous craft are physically unreachable — onboard watchdogs are the last line of defense.

EVALUATION

Advantages & Limitations

Advantages

- Automatic recovery from software hangs — no human intervention needed
- Significantly improves long-term system reliability & uptime
- Catches faults regular software checks cannot (deadlocks, infinite loops)
- Very low hardware cost and negligible power overhead
- Simple to implement — a handful of register writes

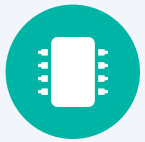
Limitations

- Resets the symptom, not the root cause — the underlying bug remains
- Careless refresh placement (e.g. inside an ISR) can mask real hangs
- Cannot detect faults where code stays "alive" but produces wrong output
- Timeout value must be tuned carefully — too short causes false resets
- Frequent unexplained resets can indicate a deeper design problem

SUMMARY

Conclusion

The Watchdog Timer is a small but essential piece of hardware: a self-monitoring safety net that keeps embedded systems reliable, autonomous, and recoverable — without which unattended electronics could freeze permanently with no way to recover.



Two flavors

IWDG for independent fault detection; WWDG for precise timing-window supervision.



Refresh discipline

Correct placement of the "kick" call is critical — it must only run when the system is truly healthy.



Safety-critical role

Standard practice across automotive, industrial, medical, IoT & aerospace embedded design.



Thank You

Questions & Discussion Welcome